VOLUMEN 02 · MÁSTER DATA & IA

SQL I

Consultar, transformar, agregar, unir y limpiar los datos reales de una empresa. Del SQL de clase al SQL de un analista.

PARA QUIÉN

Álvaro, con el Volumen 01 terminado. Viste SQL en ASIR y te defendías bien; hace tiempo que no lo tocas.

CÓMO ESTÁ ESCRITO

Cada consulta se ejecuta de verdad sobre los datos de Marejada y su resultado aparece tal cual. Lo que ya sabes se repasa rápido; lo que no se ve en clase (las trampas, la limpieza, el criterio) se explica despacio.

INCLUYE

Nueve tickets de los departamentos de Marejada con su solución y la revisión de Elena, la construcción de la capa silver y una chuleta de SQL.

Preparado para Álvaro · Octubre de 2026 · Unos tres días de trabajo

Índice

PARTE 0 · ANTES DE EMPEZAR 3

1. Cómo trabajar con este volumen 4

2. Los datos de Marejada por dentro 6

PARTE 1 · LEER Y FILTRAR 8

3. SELECT: elegir lo que ves 9

4. WHERE: filtrar filas 12

5. Ordenar, limitar y quitar repetidos 17

6. Transformar columnas 20

PARTE 2 · AGREGAR Y UNIR 26

7. Agregar 27

8. JOIN: unir tablas 32

PARTE 3 · DESCONFIAR DE LOS DATOS 38

9. Calidad de datos con SQL 39

TICKETS · UNA SEMANA EN MAREJADA 47

Tickets · Volumen 02 48

Soluciones · Volumen 02 51

PARTE 4 · LA CAPA SILVER 70

10. Construir la capa silver 71

11. Lo mismo, con silver 78

12. El mismo SQL en otros sitios 79

APÉNDICES · PARA TENER A MANO 80

A. Chuleta de SQL 81

B. Qué viene en el Volumen 03 83

PARTE 0

Antes de empezar

Cómo trabajar con este volumen y cómo son los datos que vas a consultar.

1. Cómo trabajar con este volumen

2. Los datos de Marejada por dentro

1. Cómo trabajar con este volumen

1.1 Lo que cambia respecto a ASIR

En ASIR, SQL era sobre todo una herramienta de administración: diseñar tablas, normalizarlas, crear usuarios y permisos, insertar y modificar filas. Aquí el trabajo es otro. Las tablas ya existen, las ha diseñado otra persona (o nadie), y tu trabajo es **preguntarles cosas** y **desconfiar de lo que responden**.

Ese segundo verbo es la diferencia entre el SQL de clase y el de un analista. En clase, los datos de los ejercicios están limpios y la consulta correcta da la respuesta correcta. En una empresa, una consulta perfectamente escrita puede dar un número equivocado porque los datos tienen duplicados, nombres escritos de cinco formas o fechas imposibles. Y nadie te avisa.

Por eso este volumen tiene tres partes de contenido y no dos: **leer**, **agregar y unir**, y **desconfiar**. La tercera es la que más vale.

1.2 Tu sesión de trabajo

Siempre igual: desde la carpeta del proyecto, abres la base de datos que cargaste en el Volumen 01.

TÚ ESCRIBES

```
cd ~/master-data-ia
duckdb db/marejada.duckdb
```

Para los ejemplos de los capítulos, escribe directamente en DuckDB. Para los tickets, escribe la consulta en un fichero dentro de `sql/02-sql-i/` con VS Code y ejecútala desde DuckDB con `.read`:

TÚ ESCRIBES

```
.read sql/02-sql-i/mjd-201_clientes_activos.sql
```

Así cada ticket queda guardado y en Git, que es lo que haría cualquier equipo.

> **DOS VENTANAS**
>
> Coloca VS Code a la izquierda y la Terminal con DuckDB a la derecha. Escribes en VS Code, guardas con `Cmd` + `S`, y en DuckDB pulsas `↑` e `Intro` para volver a ejecutar el `.read`. Es el ritmo de trabajo de un analista.

1.3 Cómo se escribe el SQL en estos volúmenes

SQL no distingue mayúsculas de minúsculas en sus palabras clave: `select` y `SELECT` son lo mismo. Pero hay una convención que sigue casi todo el mundo y que facilita leer las consultas:

- **Palabras de SQL en mayúsculas**: `SELECT`, `FROM`, `WHERE`, `GROUP BY`.
- **Nombres de tablas y columnas en minúsculas**: `bronze.pedidos`, `fecha_pedido`.
- **Funciones en minúsculas**: `count(*)`, `sum(...)`, `round(...)`.
- **Una cláusula por línea**, y lo que va dentro, sangrado.

Compara:

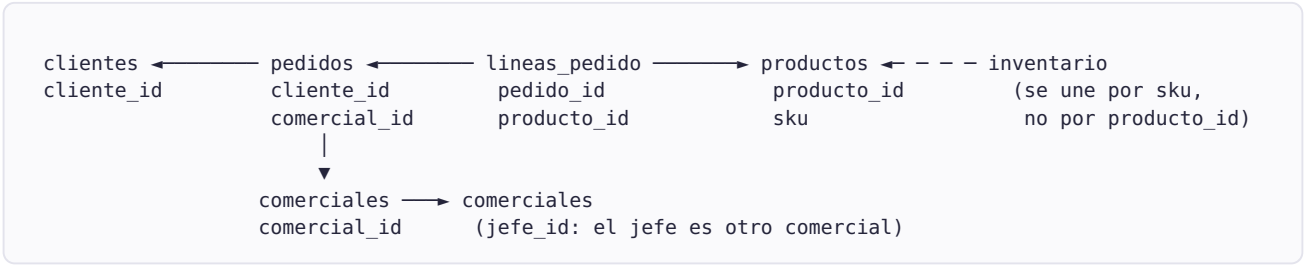
```sql
select cliente_id,count(*) from bronze.pedidos where estado='Entregado' group by cliente_id order by 2
desc limit 5
```

```sql
SELECT cliente_id,
       count(*) AS pedidos
FROM bronze.pedidos
WHERE estado = 'Entregado'
GROUP BY cliente_id
ORDER BY pedidos DESC
LIMIT 5;
```

Hacen exactamente lo mismo. La segunda se lee de un vistazo, y si mañana hay que cambiar el filtro, se ve dónde.

2. Los datos de Marejada por dentro

2.1 Las tablas y cómo se relacionan



Cada flecha sale de la tabla que tiene la columna que apunta y llega a la tabla a la que apunta. Se lee así: cada línea de pedido pertenece a un pedido y a un producto; cada pedido, a un cliente y a un comercial; cada comercial tiene un jefe que es, a su vez, otro comercial. Y el inventario se relaciona con los productos por el `sku`, no por el `producto_id`, porque viene de otro programa que no conoce los identificadores del ERP.

2.2 Las relaciones son promesas

En ASIR viste las **claves primarias** (lo que identifica cada fila, como `pedido_id`) y las **claves foráneas** (una columna que apunta a la clave primaria de otra tabla, como `cliente_id` en pedidos). En una base de datos bien diseñada, la clave foránea es una regla: la base de datos no te deja insertar un pedido con un cliente que no existe.

Pero tus tablas vienen de ficheros CSV, y un CSV no tiene reglas. En la capa bronze, que `cliente_id` apunte a un cliente real es una **promesa** del ERP, no una garantía. Y las promesas se rompen: un cliente se borra del maestro, un pedido se elimina y sus líneas se quedan. Lo comprobarás en el capítulo 9.

2.3 La pregunta más importante: qué es una fila

Antes de escribir una sola consulta sobre una tabla, hay que saber responder a esto: **¿qué representa cada fila?** Se llama la **granularidad** o el **grano** de la tabla, y de ella dependen todas las sumas que hagas.

TABLA	UNA FILA ES...
bronze.clientes	Un cliente (en teoría, uno distinto en cada fila)
bronze.productos	Un artículo del catálogo, en un formato concreto
bronze.pedidos	Un pedido completo: la cabecera
bronze.lineas_pedido	Un producto dentro de un pedido
bronze.comerciales	Una persona del equipo comercial
bronze.inventario	Un artículo en un almacén, el día del inventario

Ejemplo de por qué importa: los gastos de envío están en `pedidos`, uno por pedido. El precio está en `lineas_pedido`, uno por línea. Si sumas los gastos de envío después de juntar pedidos con sus líneas, cada pedido aparece tantas veces como líneas tenga, y los gastos salen multiplicados. Es probablemente el error más caro del oficio, y lo verás con números reales en el capítulo 8.

IDEA CLAVE

Una consulta bien escrita sobre una tabla cuyo grano no entiendes da resultados mal calculados que parecen correctos. Antes de sumar nada, pregúntate: ¿qué es una fila aquí?

PARTE 1

Leer y filtrar

SELECT, WHERE, ORDER BY y las funciones para transformar columnas. Lo que ya sabes, con las trampas que no se enseñan en clase.

3. SELECT: elegir lo que ves

4. WHERE: filtrar filas

5. Ordenar, limitar y quitar repetidos

6. Transformar columnas

3. SELECT: elegir lo que ves

3.1 Columnas y alias

SQL · EN DUCKDB

```
SELECT producto_id, sku, nombre, formato, precio_lista
FROM bronze.productos
LIMIT 5;
```

RESULTADO

producto_id	sku	nombre	formato	precio_lista
int64	varchar	varchar	varchar	double
1001	CER-0001	Faro Norte Lager	Barril 30 L	102.0
1002	CER-0002	Faro Norte Lager	Barril 20 L	70.67
1003	CER-0003	Faro Norte Lager	Caja 24x33 cl lata	20.94
1004	CER-0004	Faro Norte Lager	Caja 12x75 cl	19.2
1005	CER-0005	Faro Norte Especial	Barril 30 L	117.77

`LIMIT 5` se queda con las cinco primeras filas. Úsalo siempre que explores una tabla: con 381.000 líneas, un `SELECT *` sin límite inunda la pantalla.

Un **alias** cambia el nombre con el que sale una columna. Se escribe con `AS`:

SQL · EN DUCKDB

```
SELECT sku AS referencia,
       precio_lista AS precio,
       coste_unitario AS coste
FROM bronze.productos
LIMIT 3;
```

RESULTADO

referencia	precio	coste
varchar	double	double
CER-0001	102.0	70.33
CER-0002	70.67	51.75
CER-0003	20.94	14.58

3.2 Calcular columnas nuevas

En el `SELECT` no solo se eligen columnas: se pueden calcular. El margen de cada producto:

SQL · EN DUCKDB

```
SELECT sku,
       precio_lista,
       coste_unitario,
       precio_lista - coste_unitario AS margen,
       round(100 * (precio_lista - coste_unitario) / precio_lista, 1) AS margen_pct
FROM bronze.productos
LIMIT 5;
```

RESULTADO

sku varchar	precio_lista double	coste_unitario double	margen double	margen_pct double
CER-0001	102.0	70.33	31.67	31.0
CER-0002	70.67	51.75	18.92	26.8
CER-0003	20.94	14.58	6.360000000000001	30.4
CER-0004	19.2	14.2	5.0	26.0
CER-0005	117.77	79.49	38.28	32.5

DESMONTANDO EL COMANDO

`round(100 * (precio_lista - coste_unitario) / precio_lista, 1)`

`precio_lista - coste_unitario`	El margen en euros. Los paréntesis hacen que se reste primero.
`100 * ... / precio_lista`	El margen como porcentaje del precio.
`round( ..., 1)`	Redondea a un decimal. Solo para mostrar: así no aparecen quince decimales.

3.3 El importe de una línea

El cálculo que más vas a escribir en todo el máster: cuánto vale una línea de pedido.

SQL · EN DUCKDB

```
SELECT pedido_id, num_linea, cantidad, precio_unitario, descuento_pct,
       cantidad * precio_unitario * (1 - descuento_pct / 100) AS importe_neto
FROM bronze.lineas_pedido
LIMIT 5;
```

RESULTADO

pedido_id int64	num_linea int64	cantidad int64	precio_unitario double	descuento_pct int64	importe_neto double
2024000001	1	6	41.65	0	249.89999999999998
2024000001	2	2	95.85	0	191.7
2024000002	1	3	95.49	15	243.49949999999995
2024000002	2	3	44.23	0	132.69
2024000002	3	4	44.26	0	177.04

Cantidad por precio, menos el descuento. Fíjate en `descuento_pct / 100`: el descuento viene en porcentaje (15), y para restarlo hay que convertirlo en proporción (0,15). `1 - 0,15 = 0,85`: pagas el 85 %.

LA DIVISIÓN DEPENDE DEL MOTOR

En DuckDB, `15 / 100` da `0.15`. En SQL Server, que es el SQL del Warehouse de Fabric, `15 / 100` entre dos enteros da `0`, porque divide como enteros y descarta los decimales. Y el descuento se convertiría en cero sin avisar. Lo verás en el capítulo 12. La costumbre segura en cualquier motor: `descuento_pct / 100.0`.

3.4 Todas las columnas menos algunas

`SELECT *` trae todas las columnas. DuckDB añade un atajo muy útil para cuando quieres casi todas:

SQL · EN DUCKDB

```
SELECT * EXCLUDE (razon_social, telefono, email, limite_credito, descuento_habitual)
FROM bronze.clientes
LIMIT 3;
```

RESULTADO

```
┌────────────┬─────────────┬──────────────────┬───┬──────────────┬──────────────┬─────────┐
│ cliente_id │     nif     │ nombre_comercial │ … │ comercial_id │ fecha_alta   │ activo  │
│   int64    │   varchar   │     varchar      │ … │    int64     │    date      │ varchar │
├────────────┼─────────────┼──────────────────┼───┼──────────────┼──────────────┼─────────┤
│      10001 │ B39007919   │ Bar La Ola       │ … │            5 │ 2024-11-26   │ S       │
│      10002 │ B-39015838  │ Bar La Ría       │ … │            2 │ 2012-02-25   │ S       │
│      10003 │ A39023757   │ Bar Bitácora     │ … │            8 │ 2019-12-12   │ S       │
└────────────┴─────────────┴──────────────────┴───┴──────────────┴──────────────┴─────────┘
  3 rows                                                  10 columns (6 shown)
```

`EXCLUDE` no es SQL estándar: existe en DuckDB y en Databricks, pero no en SQL Server. Úsalo para explorar; en las consultas que entregas, escribe la lista de columnas que quieres. Así quien la lea sabe exactamente qué sale.

4. WHERE: filtrar filas

4.1 Comparaciones

SQL · EN DUCKDB

```
SELECT count(*) AS pedidos_cancelados
FROM bronze.pedidos
WHERE estado = 'Cancelado';
```

RESULTADO

```
┌──────────────────┐
│ pedidos_cancelados │
│       int64        │
├──────────────────┤
│             2732 │
└──────────────────┘
```

Los operadores son los de siempre: `=`, `<>` (distinto; también vale `!=`), `<`, `>`, `<=`, `>=`. Los textos, entre comillas simples, y escritos **exactamente** como están en los datos: `'cancelado'` en minúsculas no encontraría nada.

4.2 AND, OR y los paréntesis

Raquel quiere los pedidos de teléfono o de comercial que se hayan cancelado. Primer intento:

SQL · EN DUCKDB

```
SELECT count(*) AS pedidos
FROM bronze.pedidos
WHERE canal = 'Teléfono' OR canal = 'Comercial' AND estado = 'Cancelado';
```

RESULTADO

```
┌─────────┐
│ pedidos │
│  int64  │
├─────────┤
│   22595 │
└─────────┘
```

Más de 22.000 pedidos cancelados, de 77.000. Demasiados: sería casi un 30 %. Lo que ha pasado es que `AND` tiene **prioridad** sobre `OR`, igual que la multiplicación sobre la suma. DuckDB lo ha leído así:

SQL

```
WHERE canal = 'Teléfono' OR (canal = 'Comercial' AND estado = 'Cancelado')
```

Es decir: **todos** los pedidos de teléfono, cancelados o no, más los cancelados de comercial. Con paréntesis:

SQL · EN DUCKDB

```
SELECT count(*) AS pedidos
FROM bronze.pedidos
WHERE (canal = 'Teléfono' OR canal = 'Comercial') AND estado = 'Cancelado';
```

RESULTADO

```
┌──────────┐
│ pedidos  │
│  int64   │
├──────────┤
│     2435 │
└──────────┘
```

IDEA CLAVE

Cuando mezcles `AND` y `OR`, pon paréntesis siempre, aunque creas que no hacen falta. Es el mismo error de la prioridad de operaciones de primaria, y da resultados que parecen razonables.

4.3 IN: una lista de valores

La versión limpia de lo anterior:

SQL · EN DUCKDB

```
SELECT count(*) AS pedidos
FROM bronze.pedidos
WHERE canal IN ('Teléfono', 'Comercial')
  AND estado = 'Cancelado';
```

RESULTADO

```
┌──────────┐
│ pedidos  │
│  int64   │
├──────────┤
│     2435 │
└──────────┘
```

`IN (...)` significa "igual a cualquiera de estos". Es más corto que encadenar `OR` y elimina el problema de los paréntesis.

4.4 La trampa de las fechas con hora

¿Cuántos pedidos se hicieron en marzo de 2025? La forma que parece natural:

SQL · EN DUCKDB

```
SELECT count(*) AS pedidos
FROM bronze.pedidos
WHERE fecha_pedido BETWEEN '2025-03-01' AND '2025-03-31';
```

RESULTADO

```
┌──────────┐
│ pedidos  │
│  int64   │
├──────────┤
│     1823 │
└──────────┘
```

Y la forma correcta:

SQL · EN DUCKDB

```
SELECT count(*) AS pedidos
FROM bronze.pedidos
WHERE fecha_pedido >= '2025-03-01'
  AND fecha_pedido <  '2025-04-01';
```

RESULTADO

```
┌──────────┐
│ pedidos  │
│  int64   │
├──────────┤
│     1908 │
└──────────┘
```

Faltan 85 pedidos en la primera. ¿Dónde están? En el 31 de marzo. `fecha_pedido` es de tipo `TIMESTAMP`: fecha **y hora**. Cuando la comparas con `'2025-03-31'`, DuckDB convierte ese texto en `2025-03-31 00:00:00`, la medianoche con la que **empieza** el día 31. Así que `BETWEEN` se queda con lo anterior a esa medianoche y deja fuera todo lo pedido el día 31 a partir de las 00:00.

> **IDEA CLAVE**
>
> Para filtrar un periodo, usa siempre "mayor o igual que el primer día" y "menor que el primer día del periodo siguiente". Funciona igual si la columna tiene hora o no, y no tienes que saber cuántos días tiene el mes.

4.5 Buscar dentro de un texto

`LIKE` busca patrones: `%` es "cualquier texto, de cualquier longitud" y `_` es "un carácter cualquiera".

SQL · EN DUCKDB

```
SELECT cliente_id, nombre_comercial, municipio
FROM bronze.clientes
WHERE nombre_comercial LIKE '%Puerto%'
LIMIT 5;
```

RESULTADO

```
┌────────────┬──────────────────────┬──────────────────────┐
│ cliente_id │   nombre_comercial   │      municipio       │
│   int64    │       varchar        │       varchar        │
├────────────┼──────────────────────┼──────────────────────┤
│      10034 │ Restaurante El Puerto │ Guardo               │
│      10093 │ Cafetería El Puerto   │ Ribamontán al Mar    │
│      10125 │ Restaurante El Puerto │ Camargo              │
│      10148 │ Bar El Puerto         │ Santa Cruz de Bezana │
│      10150 │ Sidrería El Puerto    │ GIJÓN                │
└────────────┴──────────────────────┴──────────────────────┘
```

`LIKE` distingue mayúsculas: no encontraría un `BAR EL PUERTO`. Para eso, `ILIKE` (la `I` es de *insensitive*):

SQL · EN DUCKDB

```
SELECT count(*) FILTER (WHERE nombre_comercial LIKE '%puerto%')  AS con_like,
       count(*) FILTER (WHERE nombre_comercial ILIKE '%puerto%') AS con_ilike
FROM bronze.clientes;
```

RESULTADO

```
┌──────────┬────────────┐
│ con_like │ con_ilike  │
│  int64   │   int64    │
├──────────┼────────────┤
│        0 │         20 │
└──────────┴────────────┘
```

Con `LIKE '%puerto%'`, en minúsculas, no aparece ninguno: todos lo escriben con mayúscula. Con `ILIKE`, aparecen todos, se escriban como se escriban. (El `count(*) FILTER (...)` es un truco para contar dos cosas en la misma consulta; lo verás en el capítulo 7.)

4.6 NULL: el valor que no es un valor

`NULL` significa "aquí no hay dato". No es cero ni un texto vacío: es ausencia. Y se comporta de una forma que sorprende. ¿Cuántos clientes no tienen email?

SQL · EN DUCKDB

```
SELECT count(*) AS sin_email
FROM bronze.clientes
WHERE email = NULL;
```

RESULTADO

```
┌───────────┐
│ sin_email │
│   int64   │
├───────────┤
│         0 │
└───────────┘
```

Cero. Y es falso: hay muchos. Ahora así:

SQL · EN DUCKDB

```
SELECT count(*) AS sin_email
FROM bronze.clientes
WHERE email IS NULL;
```

RESULTADO

```
┌───────────┐
│ sin_email │
│   int64   │
├───────────┤
│       190 │
└───────────┘
```

La explicación es la clave de todo `NULL`: cualquier comparación con `NULL` da `NULL`, no verdadero ni falso. Si no sabes cuál es el email de un cliente, tampoco puedes saber si es igual a "nada": la respuesta es "no se sabe". Y `WHERE` solo deja pasar las filas en las que la condición es verdadera. Por eso existe `IS NULL`, que sí responde verdadero o falso.

EXPRESIÓN	RESULTADO
`NULL = NULL`	NULL (no se sabe)
`NULL <> 5`	NULL
`NULL + 10`	NULL
`email IS NULL`	true o false
`email IS NOT NULL`	true o false

IDEA CLAVE

Con nulos, nunca `= NULL`: siempre `IS NULL` o `IS NOT NULL`. Y recuerda que cualquier operación con un `NULL` dentro da `NULL`: un precio desconocido por una cantidad da un importe desconocido.

5. Ordenar, limitar y quitar repetidos

5.1 ORDER BY

SQL · EN DUCKDB

```
SELECT sku, nombre, formato, precio_lista
FROM bronze.productos
ORDER BY precio_lista DESC
LIMIT 5;
```

RESULTADO

```
┌──────────┬──────────────────────┬──────────────┬──────────────┐
│   sku    │        nombre        │   formato    │ precio_lista │
│ varchar  │       varchar        │   varchar    │    double    │
├──────────┼──────────────────────┼──────────────┼──────────────┤
│ CER-0005 │ Faro Norte Especial  │ Barril 30 L  │       117.77 │
│ CER-0055 │ Ola Fría Especial    │ Barril 30 L  │       116.55 │
│ CER-0042 │ Pejina Tostada       │ Barril 30 L  │       114.93 │
│ CER-0039 │ Pejina Especial      │ Barril 30 L  │       113.09 │
│ CER-0007 │ Faro Norte Tostada   │ Barril 30 L  │       106.97 │
└──────────┴──────────────────────┴──────────────┴──────────────┘
```

`DESC` es de mayor a menor; `ASC` (o nada) de menor a mayor. Se puede ordenar por varias columnas: `ORDER BY categoria, precio_lista DESC` ordena por categoría y, dentro de cada una, por precio.

Y otra vez los nulos: ¿dónde van los productos sin fecha de baja al ordenar por ella? Depende del motor. Para no depender, se dice:

SQL · EN DUCKDB

```
SELECT sku, activo, fecha_baja
FROM bronze.productos
ORDER BY fecha_baja DESC NULLS LAST
LIMIT 4;
```

RESULTADO

```
┌──────────┬──────────┬────────────┐
│   sku    │ activo   │ fecha_baja │
│ varchar  │ varchar  │    date    │
├──────────┼──────────┼────────────┤
│ CAF-0011 │ N        │ 2026-04-14 │
│ CER-0026 │ N        │ 2026-04-01 │
│ VIN-0024 │ N        │ 2026-03-01 │
│ LIC-0005 │ N        │ 2025-11-20 │
└──────────┴──────────┴────────────┘
```

5.2 DISTINCT

`DISTINCT` quita las filas repetidas del resultado. ¿Qué estados de pedido existen?

SQL · EN DUCKDB

```
SELECT DISTINCT estado
FROM bronze.pedidos
ORDER BY estado;
```

RESULTADO

```
┌──────────────────┐
│      estado      │
│     varchar      │
├──────────────────┤
│ Cancelado        │
│ Devuelto parcial │
│ En reparto       │
│ Entregado        │
│ Pendiente        │
└──────────────────┘
```

Es la forma más rápida de conocer los valores de una columna de texto. Hazlo con todas las columnas de categorías cuando conozcas una tabla nueva: es como se descubren los problemas. Prueba con la provincia:

SQL · EN DUCKDB

```
SELECT DISTINCT provincia
FROM bronze.clientes
ORDER BY provincia;
```

RESULTADO

```
┌──────────────────────────┐
│        provincia         │
│         varchar          │
├──────────────────────────┤
│ ASTURIAS                 │
│ Asturias                 │
│ BIZKAIA                  │
│ BURGOS                   │
│ Bizkaia                  │
│ Burgos                   │
│ CANTABRIA                │
│ Cantabria                │
│ Cantabria                │
│ Palencia                 │
│ Principado de Asturias   │
│ Santander                │
│ Vizcaya                  │
│ cantabria                │
│ palencia                 │
└──────────────────────────┘
          15 rows
```

Quince formas de escribir cinco provincias. Fíjate en las dos `Cantabria` que parecen iguales: una lleva un espacio al final que no se ve. Para DuckDB son textos distintos. Lo arreglarás en el capítulo 9.

5.3 El orden en que se ejecuta una consulta

Escribes las cláusulas en un orden, pero DuckDB las ejecuta en otro. Entenderlo explica muchos errores:

ORDEN DE EJECUCIÓN	CLÁUSULA	QUÉ HACE
1	FROM (y los JOIN)	Elige las tablas de las que salen los datos.
2	WHERE	Filtra filas.

ORDEN DE EJECUCIÓN	CLÁUSULA	QUÉ HACE
3	GROUP BY	Agrupa.
4	HAVING	Filtra grupos.
5	SELECT	Calcula las columnas de salida y les pone nombre.
6	DISTINCT	Quita repetidos.
7	ORDER BY	Ordena.
8	LIMIT	Corta.

Por ejemplo: un alias creado en el `SELECT` se puede usar en el `ORDER BY` (que va después), pero en SQL estándar no en el `WHERE` (que va antes). DuckDB es más permisivo y a veces lo deja, pero SQL Server no. Si te acostumbras a la regla, tu SQL funcionará en todas partes.

6. Transformar columnas

6.1 Texto

SQL · EN DUCKDB

```
SELECT '  Cantabria ' AS original,
       trim('  Cantabria ')  AS sin_espacios,
       upper('Cantabria')    AS mayusculas,
       lower('Cantabria')    AS minusculas,
       length('Cantabria')   AS letras;
```

RESULTADO

```
┌──────────────┬──────────────┬─────────────┬──────────────┬─────────┐
│   original   │ sin_espacios │ mayusculas  │ minusculas   │ letras  │
│   varchar    │   varchar    │  varchar    │  varchar     │ int64   │
├──────────────┼──────────────┼─────────────┼──────────────┼─────────┤
│   Cantabria  │ Cantabria    │ CANTABRIA   │ cantabria    │       9 │
└──────────────┴──────────────┴─────────────┴──────────────┴─────────┘
```

FUNCIÓN	QUÉ HACE	EJEMPLO → RESULTADO
`trim(t)`	Quita los espacios del principio y del final	`trim(' Bar ')` → `'Bar'`
`upper(t)` / `lower(t)`	Mayúsculas / minúsculas	`lower('BAR')` → `'bar'`
`replace(t, a, b)`	Cambia todas las `a` por `b`	`replace('B-39', '-', '')` → `'B39'`
`length(t)`	Número de caracteres	`length('CER-0001')` → `8`
`a		b`	Concatena (une) dos textos	`'CER'	'-'	'0001'` → `'CER-0001'`
`left(t, n)` / `right(t, n)`	Los n primeros / últimos caracteres	`right('+34942000000', 9)` → `'942000000'`

FUNCIÓN	QUÉ HACE	EJEMPLO → RESULTADO
`split_part(t, sep, n)`	El trozo número n al partir por un separador	`split_part('CER-0001', '-', 1)` → `'CER'`
`strip_accents(t)`	Quita tildes	`strip_accents('Alimentación')` → `'Alimentacion'`
`lpad(t, n, c)`	Rellena por la izquierda hasta n caracteres	`lpad('9001', 5, '0')` → `'09001'`

Las funciones se anidan: se ejecutan de dentro hacia fuera. El NIF de un cliente, normalizado:

SQL · EN DUCKDB

```
SELECT nif,
       upper(replace(trim(nif), '-', '')) AS nif_normalizado
FROM bronze.clientes
WHERE nif LIKE '%-%' OR nif <> upper(nif) OR nif LIKE ' %'
LIMIT 5;
```

RESULTADO

```
┌──────────────┬──────────────────┐
│     nif      │ nif_normalizado  │
│   varchar    │     varchar      │
├──────────────┼──────────────────┤
│ B-39015838   │ B39015838        │
│ B-39079190   │ B39079190        │
│ J-39126704   │ J39126704        │
│ B-39150461   │ B39150461        │
│ B-39308841   │ B39308841        │
└──────────────┴──────────────────┘
```

Primero `trim` quita espacios, luego `replace` quita guiones y por último `upper` pone mayúsculas. Tres formas distintas de escribir lo mismo acaban siendo una.

6.2 Números y conversiones de tipo

SQL · EN DUCKDB

```
SELECT 7 / 2         AS division,
       7 // 2        AS division_entera,
       7 % 2         AS resto,
       round(2.675, 2) AS redondeo,
       CAST('42' AS INTEGER) AS texto_a_numero,
       '42'::INTEGER AS lo_mismo_abreviado;
```

RESULTADO

division double	division_entera int32	resto int32	redondeo decimal(4,2)	texto_a_numero int32	lo_mismo_abreviado int32
3.5	3	1	2.68	42	42

`CAST(x AS tipo)` convierte un valor a otro tipo. `x::tipo` es la forma abreviada (vale en DuckDB y en Databricks, no en SQL Server). Si la conversión es imposible, da error:

SQL · EN DUCKDB

```
SELECT CAST('cuarenta' AS INTEGER);
```

ERROR

```
Conversion Error: Could not convert string 'cuarenta' to INT32
LINE 1: SELECT CAST('cuarenta' AS INTEGER)
               ^
```

Cuando no sabes si todos los valores de una columna se pueden convertir, `TRY_CAST` devuelve `NULL` en lugar de fallar:

SQL · EN DUCKDB

```
SELECT TRY_CAST('42' AS INTEGER) AS bien,
       TRY_CAST('cuarenta' AS INTEGER) AS mal;
```

RESULTADO

bien int32	mal int32
42	NULL

6.3 Fechas

Las fechas tienen sus propias funciones, y son de las más usadas:

SQL · EN DUCKDB

```
SELECT fecha_pedido,
       CAST(fecha_pedido AS DATE)         AS dia,
       year(fecha_pedido)                 AS anio,
       month(fecha_pedido)                AS mes,
       quarter(fecha_pedido)              AS trimestre,
       date_trunc('month', fecha_pedido)  AS inicio_mes,
       strftime(fecha_pedido, '%Y-%m')    AS mes_texto
FROM bronze.pedidos
LIMIT 3;
```

RESULTADO

fecha_pedido timestamp	dia date	anio int64	mes int64	trimestre int64	inicio_mes timestamp	mes_texto varchar
2024-01-01 08:03:00	2024-01-01	2024	1	1	2024-01-01 00:00:00	2024-01
2024-01-01 08:21:00	2024-01-01	2024	1	1	2024-01-01 00:00:00	2024-01
2024-01-01 08:23:00	2024-01-01	2024	1	1	2024-01-01 00:00:00	2024-01

`date_trunc('month', ...)` "recorta" la fecha al principio de su mes: todos los pedidos de marzo quedan en `2025-03-01`. Es la forma estándar de agrupar por mes, porque el resultado sigue siendo una fecha y se ordena bien. `strftime` convierte una fecha en texto con el formato que quieras (las letras con `%` del Volumen 01).

Restar fechas da días, y sumar un intervalo mueve una fecha:

SQL · EN DUCKDB

```
SELECT pedido_id,
       fecha_pedido,
       fecha_entrega,
       fecha_entrega - CAST(fecha_pedido AS DATE) AS dias_entrega,
       fecha_entrega + INTERVAL 30 DAY            AS vencimiento_factura
FROM bronze.pedidos
WHERE estado = 'Entregado'
LIMIT 3;
```

RESULTADO

pedido_id int64	fecha_pedido timestamp	fecha_entrega date	dias_entrega int64	vencimiento_factura timestamp
2024000001	2024-01-01 08:03:00	2024-01-04	3	2024-02-03 00:00:00
2024000002	2024-01-01 08:21:00	2024-01-02	1	2024-02-01 00:00:00
2024000003	2024-01-01 08:23:00	2024-01-03	2	2024-02-02 00:00:00

LA FECHA DE HOY EN MAREJADA

Para preguntas como "los últimos 90 días", la tentación es usar `current_date`, la fecha de hoy. Pero los datos de Marejada llegan hasta el 30 de septiembre de 2026, y "hoy" para tu ordenador es otro día: el análisis saldría distinto cada vez que lo ejecutes. En análisis de datos se usa una **fecha de corte** fija y escrita en la consulta: `DATE '2026-09-30'`. Así el resultado es reproducible.

LOS DÍAS DE LA SEMANA EN INGLÉS

`dayname(fecha)` devuelve el nombre del día en inglés: `Monday`. Para agrupar por día de la semana usa `isodow(fecha)`, que devuelve un número del 1 (lunes) al 7 (domingo), y tradúcelo con un `CASE` si hace falta mostrarlo.

6.4 CASE: decisiones dentro de una consulta

`CASE` es el `if` de SQL. Clasifica cada pedido por su tamaño:

SQL · EN DUCKDB

```
SELECT pedido_id,
       gastos_envio,
       CASE
           WHEN gastos_envio = 0  THEN 'Portes pagados'
           WHEN gastos_envio = 12 THEN 'Portes Cantabria'
           ELSE 'Portes fuera de Cantabria'
       END AS tipo_portes
FROM bronze.pedidos
LIMIT 5;
```

RESULTADO

```
┌────────────┬──────────────┬──────────────────┐
│ pedido_id  │ gastos_envio │   tipo_portes    │
│   int64    │    double    │     varchar      │
├────────────┼──────────────┼──────────────────┤
│ 2024000001 │          0.0 │ Portes pagados   │
│ 2024000002 │          0.0 │ Portes pagados   │
│ 2024000003 │          0.0 │ Portes pagados   │
│ 2024000004 │          0.0 │ Portes pagados   │
│ 2024000005 │          0.0 │ Portes pagados   │
└────────────┴──────────────┴──────────────────┘
```

DESMONTANDO LA CONSULTA

`CASE WHEN ... THEN ... ELSE ... END AS tipo_portes`

`CASE`	Empieza la decisión.
`WHEN condición THEN valor`	Si se cumple la condición, el resultado es ese valor. Se comprueban en orden y gana la primera que se cumpla, como el `else if` de JavaScript.
`ELSE valor`	Si no se cumple ninguna. Si no pones `ELSE`, el resultado es `NULL`.
`END`	Termina la decisión. Todo el bloque es una sola columna.
`AS tipo_portes`	Su nombre.

6.5 COALESCE y NULLIF

`coalesce(a, b, c...)` devuelve el primer valor que no sea `NULL`. Sirve para poner un valor por defecto:

SQL · EN DUCKDB

```
SELECT nombre_comercial,
       email,
       coalesce(email, '(sin email)') AS email_para_listado
FROM bronze.clientes
WHERE email IS NULL
LIMIT 3;
```

RESULTADO

nombre_comercial	email	email_para_listado
varchar	varchar	varchar
Bar La Barquera	NULL	(sin email)
Supermercado La Pradera	NULL	(sin email)
Sidrería El Sardinero	NULL	(sin email)

`nullif(a, b)` hace lo contrario: devuelve `NULL` si `a` es igual a `b`. Su uso más común es evitar dividir entre cero: `ventas / nullif(pedidos, 0)` da `NULL` en lugar de un error si no hay pedidos.

PARTE 2

Agregar y unir

Resumir miles de filas en unas pocas cifras, y juntar tablas sin que las cifras salgan multiplicadas.

7. Agregar

8. JOIN: unir tablas

7. Agregar

7.1 Las funciones de agregado

Una función de agregado convierte muchas filas en un solo valor:

SQL · EN DUCKDB

```
SELECT count(*)                AS lineas,
       sum(cantidad)           AS unidades,
       round(avg(cantidad), 2) AS media_unidades,
       min(precio_unitario)    AS precio_minimo,
       max(precio_unitario)    AS precio_maximo
FROM bronze.lineas_pedido;
```

RESULTADO

```
┌─────────┬──────────┬──────────────────┬─────────────────┬─────────────────┐
│ lineas  │ unidades │ media_unidades   │ precio_minimo   │ precio_maximo   │
│ int64   │  int128  │     double       │    double       │    double       │
├─────────┼──────────┼──────────────────┼─────────────────┼─────────────────┤
│ 381158  │  1974952 │           5.18   │           3.5   │        117.77   │
└─────────┴──────────┴──────────────────┴─────────────────┴─────────────────┘
```

7.2 Los tres count

`count` tiene tres versiones, y confundirlas es de los errores más comunes:

SQL · EN DUCKDB

```
SELECT count(*)                  AS filas,
       count(email)              AS con_email,
       count(DISTINCT tipo_cliente) AS tipos_distintos
FROM bronze.clientes;
```

RESULTADO

```
┌─────────┬────────────┬─────────────────┐
│ filas   │ con_email  │ tipos_distintos │
│ int64   │   int64    │      int64      │
├─────────┼────────────┼─────────────────┤
│  1212   │      1022  │               9 │
└─────────┴────────────┴─────────────────┘
```

FORMA	CUENTA	AQUÍ
`count(*)`	Filas, sin mirar ninguna columna	Todos los clientes
`count(columna)`	Filas en las que esa columna **no** es `NULL`	Los que tienen email
`count(DISTINCT columna)`	Valores distintos (sin contar `NULL`)	Cuántos tipos de cliente hay

La pregunta "¿cuántos clientes tienen email?" se responde con `count(email)`; "¿cuántos clientes hay?", con `count(*)`. La diferencia entre las dos cifras son los nulos.

7.3 GROUP BY

`GROUP BY` hace los cálculos por grupos en lugar de para toda la tabla. Pedidos por canal:

SQL · EN DUCKDB

```
SELECT canal,
       count(*) AS pedidos
FROM bronze.pedidos
GROUP BY canal
ORDER BY pedidos DESC;
```

RESULTADO

```
┌────────────┬──────────┐
│   canal    │ pedidos  │
│  varchar   │  int64   │
├────────────┼──────────┤
│ Comercial  │    48612 │
│ Teléfono   │    20911 │
│ Web B2B    │     7925 │
└────────────┴──────────┘
```

Y la regla que no se puede romper: **en el SELECT, toda columna que no esté dentro de una función de agregado tiene que estar en el GROUP BY**. Si no:

SQL · EN DUCKDB

```
SELECT canal, estado, count(*) AS pedidos
FROM bronze.pedidos
GROUP BY canal;
```

ERROR

```
Binder Error: column "estado" must appear in the GROUP BY clause or must be part of an aggregate
function.
Either add it to the GROUP BY list, or use "ANY_VALUE(estado)" if the exact value of "estado" is not
important.
```

Tiene toda la lógica: has pedido un grupo por canal, y dentro del grupo "Teléfono" hay pedidos entregados, cancelados, pendientes... ¿Qué `estado` debería mostrar la fila de "Teléfono"? No hay respuesta, así que DuckDB se niega. O agrupas también por estado, o lo metes en un agregado.

DuckDB tiene un atajo muy práctico: `GROUP BY ALL` agrupa automáticamente por todas las columnas que no son agregados.

SQL · EN DUCKDB

```
SELECT canal, estado, count(*) AS pedidos
FROM bronze.pedidos
GROUP BY ALL
ORDER BY canal, pedidos DESC;
```

RESULTADO

canal varchar	estado varchar	pedidos int64
Comercial	Entregado	46030
Comercial	Cancelado	1684
Comercial	Devuelto parcial	771
Comercial	Pendiente	69
Comercial	En reparto	58
Teléfono	Entregado	19814
Teléfono	Cancelado	751
Teléfono	Devuelto parcial	286
Teléfono	En reparto	33
Teléfono	Pendiente	27
Web B2B	Entregado	7436
Web B2B	Cancelado	297
Web B2B	Devuelto parcial	107
Web B2B	En reparto	49
Web B2B	Pendiente	36

15 rows 3 columns

Existe también en Databricks, pero no en SQL Server. Para los entregables, prefiere escribir las columnas.

7.4 WHERE frente a HAVING

Los dos filtran, pero en momentos distintos (vuelve a la tabla de 5.3): `WHERE` filtra **filas antes** de agrupar; `HAVING` filtra **grupos después** de agrupar. Clientes con más de 200 pedidos entregados:

SQL · EN DUCKDB

```
SELECT cliente_id,
       count(*) AS pedidos_entregados
FROM bronze.pedidos
WHERE estado = 'Entregado'
GROUP BY cliente_id
HAVING count(*) > 200
ORDER BY pedidos_entregados DESC;
```

RESULTADO

cliente_id int64	pedidos_entregados int64
11108	340
10346	232
11057	215
10769	205

El `WHERE` descarta los pedidos no entregados antes de contar. El `HAVING` descarta, después de contar, los clientes que no llegan a 200. No se podría escribir `WHERE count(*) > 200`, porque en el momento del `WHERE` todavía no se ha contado nada.

7.5 Contar varias cosas a la vez: FILTER

Lo viste en 4.5. `FILTER (WHERE ...)` aplica una condición a un solo agregado, y permite calcular muchas cifras en una sola pasada por la tabla:

SQL · EN DUCKDB

```
SELECT year(fecha_pedido) AS anio,
       count(*) AS pedidos,
       count(*) FILTER (WHERE estado = 'Cancelado') AS cancelados,
       round(100.0 * count(*) FILTER (WHERE estado = 'Cancelado') / count(*), 2) AS pct_cancelados
FROM bronze.pedidos
GROUP BY anio
ORDER BY anio;
```

RESULTADO

```
┌─────────┬─────────┬─────────────┬─────────────────┐
│ anio    │ pedidos │ cancelados  │ pct_cancelados  │
│ int64   │  int64  │   int64     │     double      │
├─────────┼─────────┼─────────────┼─────────────────┤
│  2024   │   24048 │         837 │            3.48 │
│  2025   │   28234 │        1014 │            3.59 │
│  2026   │   25166 │         881 │             3.5 │
└─────────┴─────────┴─────────────┴─────────────────┘
```

En motores que no tienen `FILTER` (SQL Server), lo mismo se escribe con `CASE` dentro del agregado: `sum(CASE WHEN estado = 'Cancelado' THEN 1 ELSE 0 END)`. Es más largo, pero funciona en todas partes.

7.6 Facturación por mes

Ya tienes todas las piezas para la primera cifra de negocio: facturación (ventas netas) por mes en 2025.

SQL · EN DUCKDB

```
SELECT date_trunc('month', p.fecha_pedido)::DATE AS mes,
       round(sum(l.cantidad * l.precio_unitario * (1 - l.descuento_pct / 100)), 2) AS ventas_netas
FROM bronze.pedidos AS p
JOIN bronze.lineas_pedido AS l ON l.pedido_id = p.pedido_id
WHERE p.estado <> 'Cancelado'
  AND p.fecha_pedido >= '2025-01-01'
  AND p.fecha_pedido <  '2026-01-01'
GROUP BY mes
ORDER BY mes;
```

RESULTADO

```
┌────────────┬──────────────┐
│    mes     │ ventas_netas │
│    date    │    double    │
├────────────┼──────────────┤
│ 2025-01-01 │    976488.84 │
│ 2025-02-01 │     850868.9 │
│ 2025-03-01 │   1066085.78 │
│ 2025-04-01 │   1223343.05 │
│ 2025-05-01 │   1260454.31 │
│ 2025-06-01 │   1357013.38 │
│ 2025-07-01 │   1722203.51 │
│ 2025-08-01 │   1676552.29 │
│ 2025-09-01 │   1477232.19 │
│ 2025-10-01 │   1292071.37 │
│ 2025-11-01 │   1122207.68 │
│ 2025-12-01 │    1631470.2 │
└────────────┴──────────────┘
  12 rows         2 columns
```

Fíjate en el verano: julio y agosto son los mejores meses, como corresponde a una distribuidora de hostelería en la costa, y diciembre se acerca por las comidas de Navidad. Comprobar que una cifra tiene sentido con lo que sabes del negocio es parte del trabajo.

Esa consulta usa un `JOIN`, que es lo siguiente. Y tiene un problema que todavía no se ve: lo encontrarás en el capítulo 9.

DINERO EN DOUBLE

Los precios están cargados como `DOUBLE`, un tipo numérico rápido pero aproximado (es el `0.1 + 0.2` del Volumen 1). Para sumar cientos de miles de importes, el error acumulado es de fracciones de céntimo, y se resuelve redondeando **al final**, no en cada línea. En la capa silver los convertirás a `DECIMAL`, el tipo exacto para dinero.

8. JOIN: unir tablas

8.1 Por qué hace falta

Los pedidos tienen `cliente_id`, pero el nombre del cliente está en otra tabla. Juntar las dos es lo que hace un `JOIN`:

SQL · EN DUCKDB

```
SELECT p.pedido_id,
       p.fecha_pedido,
       c.nombre_comercial,
       c.municipio
FROM bronze.pedidos AS p
JOIN bronze.clientes AS c ON c.cliente_id = p.cliente_id
LIMIT 5;
```

RESULTADO

pedido_id int64	fecha_pedido timestamp	nombre_comercial varchar	municipio varchar
2024000001	2024-01-01 08:03:00	Alimentación El Paseo	Santander
2024000002	2024-01-01 08:21:00	Restaurante Arena	Santander
2024000003	2024-01-01 08:23:00	Cafetería La Galerna	San Vicente de la Barquera
2024000004	2024-01-01 08:25:00	Restaurante La Ribera	Gijón
2024000005	2024-01-01 08:26:00	Restaurante La Peña	Gijón

DESMONTANDO LA CONSULTA

`FROM bronze.pedidos AS p JOIN bronze.clientes AS c ON c.cliente_id = p.cliente_id`

`bronze.pedidos AS p`	La primera tabla, con un alias corto: `p`.
`JOIN bronze.clientes AS c`	La segunda tabla, con su alias `c`. `JOIN` a secas es un `INNER JOIN`.
`ON c.cliente_id = p.cliente_id`	La condición de unión: cada pedido se junta con el cliente que tiene su mismo `cliente_id`.
`p.pedido_id, c.nombre_comercial`	Con dos tablas, cada columna se escribe con el alias de su tabla delante. Si no, cuando las dos tienen una columna con el mismo nombre (`cliente_id`, `comercial_id`), DuckDB no sabría cuál quieres.

Cuando la columna se llama igual en las dos tablas, hay una forma corta: `JOIN bronze.clientes USING (cliente_id)`.

8.2 INNER JOIN y lo que se queda fuera

Un `INNER JOIN` solo devuelve las filas que encuentran pareja en las dos tablas. Lo que no la encuentra, desaparece sin avisar. Compara:

SQL · EN DUCKDB

```
SELECT count(*) AS pedidos
FROM bronze.pedidos;
```

RESULTADO

```
┌──────────┐
│ pedidos  │
│  int64   │
├──────────┤
│   77448  │
└──────────┘
```

SQL · EN DUCKDB

```
SELECT count(*) AS pedidos_con_cliente
FROM bronze.pedidos AS p
JOIN bronze.clientes AS c ON c.cliente_id = p.cliente_id;
```

RESULTADO

```
┌──────────────────────┐
│ pedidos_con_cliente  │
│        int64         │
├──────────────────────┤
│                77223 │
└──────────────────────┘
```

Se han perdido 225 pedidos. Son pedidos cuyo `cliente_id` no existe en el maestro de clientes: la promesa rota de 2.2. Si estuvieras sumando facturación, esa facturación habría desaparecido de tu informe.

8.3 LEFT JOIN: quedarse con todo lo de la izquierda

`LEFT JOIN` conserva **todas** las filas de la primera tabla. Las que no encuentran pareja aparecen con `NULL` en las columnas de la segunda:

SQL · EN DUCKDB

```
SELECT count(*) AS pedidos,
       count(c.cliente_id) AS con_cliente_en_maestro
FROM bronze.pedidos AS p
LEFT JOIN bronze.clientes AS c ON c.cliente_id = p.cliente_id;
```

RESULTADO

```
┌──────────┬──────────────────────────┐
│ pedidos  │ con_cliente_en_maestro   │
│  int64   │          int64           │
├──────────┼──────────────────────────┤
│   77448  │                 77223    │
└──────────┴──────────────────────────┘
```

Están los 77.448 pedidos, y `count(c.cliente_id)` (recuerda 7.2: no cuenta los nulos) dice cuántos encontraron cliente.

TIPO	DEVUELVE	CUÁNDO USARLO
`INNER JOIN` (o `JOIN`)	Solo las filas con pareja en las dos tablas	Cuando lo que no tiene pareja no te interesa, y lo sabes
`LEFT JOIN`	Todas las de la izquierda; las de la derecha si hay pareja	Casi siempre que la tabla de la izquierda es la que importa (los pedidos, las ventas)

TIPO	DEVUELVE	CUÁNDO USARLO
`FULL JOIN`	Todas las de las dos tablas	Para comparar dos fuentes y ver qué hay en una y no en la otra
`CROSS JOIN`	Cada fila de una con cada fila de la otra	Para generar combinaciones, como todos los meses por todas las provincias

IDEA CLAVE

Si la tabla de la izquierda son ventas, `LEFT JOIN`. Un `INNER JOIN` que pierde filas sin avisar es un informe que factura menos de lo que se ha vendido.

8.4 El anti-join: lo que NO tiene pareja

La pregunta inversa, "¿qué pedidos no tienen cliente?", se responde con un `LEFT JOIN` y quedándose solo con las filas donde la derecha salió vacía:

SQL · EN DUCKDB

```
SELECT p.cliente_id,
       count(*) AS pedidos_huerfanos,
       min(p.fecha_pedido)::DATE AS primero,
       max(p.fecha_pedido)::DATE AS ultimo
FROM bronze.pedidos AS p
LEFT JOIN bronze.clientes AS c ON c.cliente_id = p.cliente_id
WHERE c.cliente_id IS NULL
GROUP BY p.cliente_id
ORDER BY pedidos_huerfanos DESC;
```

RESULTADO

cliente_id int64	pedidos_huerfanos int64	primero date	ultimo date
10321	62	2024-01-03	2025-07-29
10680	53	2024-01-08	2025-09-08
11080	41	2024-01-17	2025-10-16
10265	38	2024-01-03	2025-04-10
10923	25	2024-01-05	2024-09-16
11186	6	2024-08-17	2025-02-05

Seis clientes que compraron, dejaron de comprar y alguien los borró del maestro. Sus pedidos siguen siendo ventas reales. A este patrón (`LEFT JOIN` + `WHERE derecha IS NULL`) se le llama **anti-join**, y es la herramienta principal para encontrar huérfanos.

8.5 Unir una tabla consigo misma

Cada comercial tiene un `jefe_id` que apunta a otro comercial de la misma tabla. Para ver el nombre del jefe, se une la tabla consigo misma, con dos alias distintos:

SQL · EN DUCKDB

```
SELECT c.nombre || ' ' || c.apellidos AS comercial,
       c.zona,
       j.nombre || ' ' || j.apellidos AS jefe
FROM bronze.comerciales AS c
LEFT JOIN bronze.comerciales AS j ON j.comercial_id = c.jefe_id
ORDER BY jefe NULLS FIRST, comercial;
```

RESULTADO

comercial	zona	jefe
varchar	varchar	varchar
Íñigo Sainz Ruiz	NULL	NULL
Adrián Suárez Fernández	Asturias	Nuria Álvarez Menéndez
Ane Etxeberria Zubiri	Bizkaia	Nuria Álvarez Menéndez
Mikel Goikoetxea Arana	Bizkaia	Nuria Álvarez Menéndez
Celia Gómez Sáiz	Cantabria Occidental	Raquel Fernández Cobo
Daniel Gutiérrez Pérez	Cantabria Centro	Raquel Fernández Cobo
Héctor Bustamante Lavín	Cantabria Occidental	Raquel Fernández Cobo
Lorena Arce Díaz	Cantabria Occidental	Raquel Fernández Cobo
Pablo Revuelta Gómez	Cantabria Oriental	Raquel Fernández Cobo
Sara Ceballos Ortiz	Cantabria Centro	Raquel Fernández Cobo
Nuria Álvarez Menéndez	Asturias	Íñigo Sainz Ruiz
Raquel Fernández Cobo	Cantabria Centro	Íñigo Sainz Ruiz
Rubén Martín Calvo	Castilla Norte	Íñigo Sainz Ruiz
Óscar Peña Revilla	Castilla Norte	Íñigo Sainz Ruiz

14 rows — 3 columns

Es la misma tabla, pero DuckDB la trata como si fueran dos: `c` en el papel de "comercial" y `j` en el de "jefe". `LEFT JOIN` porque Íñigo no tiene jefe, y con un `INNER JOIN` desaparecería.

8.6 La trampa del abanico

Este es el error del que hablábamos en 2.3, y es tan frecuente que tiene nombre: **fan-out** (abanico). ¿Cuánto se cobró en gastos de envío en los pedidos de marzo de 2025?

SQL · EN DUCKDB

```
SELECT sum(gastos_envio) AS gastos_envio
FROM bronze.pedidos
WHERE fecha_pedido >= '2025-03-01' AND fecha_pedido < '2025-04-01';
```

RESULTADO

gastos_envio
double
5076.0

Ahora imagina que esa cifra la calculas en la misma consulta en la que sumas las ventas, que necesita las líneas:

SQL · EN DUCKDB

```
SELECT round(sum(l.cantidad * l.precio_unitario * (1 - l.descuento_pct / 100)), 2) AS ventas,
       sum(p.gastos_envio) AS gastos_envio
FROM bronze.pedidos AS p
JOIN bronze.lineas_pedido AS l ON l.pedido_id = p.pedido_id
WHERE p.fecha_pedido >= '2025-03-01' AND p.fecha_pedido < '2025-04-01';
```

RESULTADO

```
┌────────────┬──────────────┐
│   ventas   │ gastos_envio │
│   double   │    double    │
├────────────┼──────────────┤
│ 1104552.29 │      11160.0 │
└────────────┴──────────────┘
```

Más del doble. Las ventas están bien, pero los gastos de envío no. Lo que ha pasado se ve con un solo pedido de tres líneas:

```
bronze.pedidos                 después del JOIN con sus 3 líneas
pedido   gastos_envio          pedido   linea   gastos_envio
A        12                    A        1       12
                               A        2       12      ← el mismo 12, repetido
                               A        3       12      ← y otra vez
sum = 12                       sum = 36
```

Al unir una cabecera con sus líneas, cada dato de la cabecera se repite en cada línea, como un abanico que se abre. Sumar algo de la cabecera después del `JOIN` lo multiplica por el número de líneas. Y no da ningún error.

La solución es **agregar cada cosa en su grano y unir después**: primero el importe de las líneas por pedido, luego juntarlo con la cabecera, donde cada pedido es una sola fila. Para escribirlo cómodamente, una herramienta nueva, `WITH`, que pone nombre a una consulta para usarla después como si fuera una tabla (la verás a fondo en el Volumen 03):

SQL · EN DUCKDB

```
WITH ventas_por_pedido AS (
    SELECT pedido_id,
           sum(cantidad * precio_unitario * (1 - descuento_pct / 100)) AS ventas
    FROM bronze.lineas_pedido
    GROUP BY pedido_id
)
SELECT round(sum(v.ventas), 2) AS ventas,
       sum(p.gastos_envio)     AS gastos_envio
FROM bronze.pedidos AS p
JOIN ventas_por_pedido AS v ON v.pedido_id = p.pedido_id
WHERE p.fecha_pedido >= '2025-03-01' AND p.fecha_pedido < '2025-04-01';
```

RESULTADO

```
┌────────────┬──────────────┐
│   ventas   │ gastos_envio │
│   double   │    double    │
├────────────┼──────────────┤
│ 1104552.29 │       5076.0 │
└────────────┴──────────────┘
```

Ahora las dos cifras están en el mismo grano (una fila por pedido) y cuadran.

IDEA CLAVE

Antes de sumar después de un JOIN, pregúntate: ¿esta columna es del grano de la tabla resultante? Si viene de la tabla "de arriba" (la cabecera) y has unido con la "de abajo" (las líneas), está repetida. Agrega cada cosa en su grano y junta después.

8.7 Claves que no encajan

El inventario se une con los productos por el SKU. Inténtalo:

SQL · EN DUCKDB

```
SELECT count(*) AS filas_inventario,
       count(p.producto_id) AS encuentran_producto
FROM bronze.inventario AS i
LEFT JOIN bronze.productos AS p ON p.sku = i.sku;
```

RESULTADO

```
┌──────────────────┬──────────────────────┐
│ filas_inventario │ encuentran_producto  │
│      int64       │        int64         │
├──────────────────┼──────────────────────┤
│              381 │                  336 │
└──────────────────┴──────────────────────┘
```

De 381 filas, 45 no encuentran su producto. Míralas:

SQL · EN DUCKDB

```
SELECT i.sku, i.descripcion
FROM bronze.inventario AS i
LEFT JOIN bronze.productos AS p ON p.sku = i.sku
WHERE p.producto_id IS NULL
ORDER BY i.sku
LIMIT 8;
```

RESULTADO

```
┌──────────┬──────────────────────────────────────────────────────┐
│   sku    │                      descripcion                     │
│ varchar  │                        varchar                       │
├──────────┼──────────────────────────────────────────────────────┤
│ ALI-0013 │ Aceite de girasol · Garrafa 5 L                      │
│ ALI-0018 │ Croissant congelado · Caja 60 uds                    │
│ CAF-0002 │ Café en grano mezcla 70/30 Tostadero del Norte · Paquete 1 kg │
│ CER-0011 │ Faro Norte IPA · Barril 20 L                         │
│ CER-0015 │ Faro Norte Sin alcohol · Caja 24x25 cl botella       │
│ CER-0040 │ Pejina Especial · Barril 20 L                        │
│ CER-0061 │ Ola Fría Tostada · Caja 12x75 cl                     │
│ CER-0081 │ Abadía de Liébana Sin alcohol · Barril 20 L          │
└──────────┴──────────────────────────────────────────────────────┘
```

El programa de almacén escribe algunos SKU con espacios, en minúsculas o sin guion. Un `JOIN` compara textos exactos, y `cer-0007` no es `CER-0007`. La solución es comparar las versiones normalizadas de las dos claves. La ves aplicada en el ticket MJD-209.

PARTE 3

Desconfiar de los datos

Cómo encontrar los problemas de calidad antes de que encuentren tus informes, y qué hacer con cada uno.

9. Calidad de datos con SQL

9. Calidad de datos con SQL

9.1 Perfilar una tabla

Perfilar es mirar una tabla columna a columna antes de usarla: cuántos nulos tiene cada una, cuántos valores distintos, el mínimo y el máximo. DuckDB lo hace todo de golpe con `SUMMARIZE`. Su resultado es muy ancho, así que nos quedamos con las columnas más útiles:

SQL · EN DUCKDB

```
SELECT column_name, column_type, min, max, approx_unique, null_percentage
FROM (SUMMARIZE bronze.pedidos);
```

RESULTADO

```
┌──────────────┬─────────────┬─────────────────────┬───┬──────────────┬─────────────────┐
│ column_name  │ column_type │         min         │ … │ approx_unique │ null_percentage │
│   varchar    │   varchar   │       varchar       │ … │     int64     │  decimal(9,2)   │
├──────────────┼─────────────┼─────────────────────┼───┼──────────────┼─────────────────┤
│ pedido_id    │ BIGINT      │ 2024000001          │ … │        75580 │            0.00 │
│ fecha_pedido │ TIMESTAMP   │ 2024-01-01 08:03:00 │ … │        71730 │            0.00 │
│ cliente_id   │ BIGINT      │ 10001               │ … │         1259 │            0.00 │
│ comercial_id │ BIGINT      │ 2                   │ … │           14 │            0.00 │
│ canal        │ VARCHAR     │ Comercial           │ … │            3 │            0.00 │
│ estado       │ VARCHAR     │ Cancelado           │ … │            5 │            0.00 │
│ fecha_entrega│ DATE        │ 1900-01-01          │ … │          789 │            3.88 │
│ almacen      │ VARCHAR     │ Guarnizo            │ … │            2 │            0.00 │
│ gastos_envio │ DOUBLE      │ 0.0                 │ … │            3 │            0.00 │
│ observaciones│ VARCHAR     │ Dejar en almacén trasero │ … │        6 │           93.98 │
└──────────────┴─────────────┴─────────────────────┴───┴──────────────┴─────────────────┘
  10 rows                                                  6 columns (5 shown)
```

Lectura rápida, columna a columna, como lo haría alguien con experiencia:

- `fecha_entrega` tiene como mínimo `1900-01-01`. Ningún pedido de una empresa fundada en el siglo XXI se entregó en 1900: es un valor de relleno de algún sistema antiguo. Problema.
- `fecha_entrega` tiene nulos: los pedidos cancelados y pendientes. Esperable.
- `observaciones` está vacía en más del 90 % de las filas. Normal en un campo de texto libre.
- `pedido_id` tiene tantos valores únicos como filas (aproximadamente): bien, es la clave.

> **IDEA CLAVE**
>
> `SUMMARIZE` es el primer comando que se ejecuta sobre una tabla nueva. Busca mínimos y máximos imposibles, nulos donde no debería haberlos y columnas de texto con más valores distintos de los razonables.

9.2 Duplicados exactos

Una línea de pedido se identifica por su pedido y su número de línea. No debería haber dos filas con la misma pareja:

SQL · EN DUCKDB

```
SELECT pedido_id, num_linea, count(*) AS veces
FROM bronze.lineas_pedido
GROUP BY pedido_id, num_linea
HAVING count(*) > 1
ORDER BY pedido_id
LIMIT 5;
```

RESULTADO

```
┌─────────────┬────────────┬────────┐
│ pedido_id   │ num_linea  │ veces  │
│   int64     │   int64    │ int64  │
├─────────────┼────────────┼────────┤
│ 2024000058  │          6 │      2 │
│ 2024000063  │          3 │      2 │
│ 2024000066  │          9 │      2 │
│ 2024000136  │          4 │      2 │
│ 2024000186  │          1 │      2 │
└─────────────┴────────────┴────────┘
```

¿Cuántas hay en total?

SQL · EN DUCKDB

```
SELECT count(*) AS lineas_repetidas
FROM (
    SELECT pedido_id, num_linea
    FROM bronze.lineas_pedido
    GROUP BY pedido_id, num_linea
    HAVING count(*) > 1
);
```

RESULTADO

```
┌─────────────────┐
│ lineas_repetidas │
│      int64       │
├─────────────────┤
│             950 │
└─────────────────┘
```

Mira un caso de cerca:

SQL · EN DUCKDB

```
SELECT *
FROM bronze.lineas_pedido
WHERE (pedido_id, num_linea) IN (
    SELECT pedido_id, num_linea
    FROM bronze.lineas_pedido
    GROUP BY pedido_id, num_linea
    HAVING count(*) > 1
)
ORDER BY pedido_id, num_linea
LIMIT 4;
```

RESULTADO

linea_id	pedido_id	num_linea	producto_id	cantidad	precio_unitario	descuento_pct
int64	int64	int64	int64	int64	double	int64
307	2024000058	6	1111	10	6.02	0
308	2024000058	6	1111	10	6.02	0
338	2024000063	3	1179	1	7.2	10
339	2024000063	3	1179	1	7.2	10

Las dos filas son idénticas en todo menos en `linea_id`, el número que el ERP asigna al guardar. Es el rastro típico de una sincronización que se ejecutó dos veces: la misma línea se insertó otra vez con un identificador nuevo. Cada una de esas líneas está contada dos veces en cualquier suma de ventas, incluida la del capítulo 7.6.

Para quitarlas, basta con elegir todas las columnas **menos** la que las diferencia, con `DISTINCT`:

SQL · EN DUCKDB

```
SELECT count(*) AS lineas_sin_duplicar
FROM (
    SELECT DISTINCT pedido_id, num_linea, producto_id, cantidad, precio_unitario, descuento_pct
    FROM bronze.lineas_pedido
);
```

RESULTADO

lineas_sin_duplicar
int64
380208

381.158 − 950 = 380.208. Cuadra.

NO TODO LO REPETIDO ES UN DUPLICADO

Dos líneas con el mismo producto en el mismo pedido podrían ser legítimas (alguien añadió más cajas después). Aquí sabemos que son duplicados porque coinciden también en `num_linea`, que el ERP numera de forma única dentro de cada pedido. Antes de quitar duplicados, define con precisión qué hace que dos filas sean "la misma", y si dudas, pregunta a quien conoce el sistema.

9.3 Duplicados lógicos

Más difíciles: filas que no son idénticas pero representan lo mismo. En el maestro de clientes, el mismo NIF escrito de formas distintas:

SQL · EN DUCKDB

```
SELECT upper(replace(trim(nif), '-', '')) AS nif_normalizado,
       count(*) AS fichas,
       string_agg(cliente_id::VARCHAR, ', ') AS ids,
       string_agg(nombre_comercial, ' | ') AS nombres
FROM bronze.clientes
GROUP BY nif_normalizado
HAVING count(*) > 1
ORDER BY nif_normalizado
LIMIT 5;
```

RESULTADO

nif_normalizado varchar	fichas int64	ids varchar	nombres varchar
A40069065	2	10135, 11187	Restaurante La Gaviota | Restaurante La Gaviota S.L.
A45081792	2	10768, 11183	Bar El Rincón | Bar-Restaurante El Rincón
A48106850	2	11150, 11185	Bar Arena Centro | Bar Arena Centro
B39015838	2	10002, 11182	Bar La Ría | BAR LA RÍA
B39498897	2	10063, 11215	Restaurante La Estación | Restaurante La Estación

`string_agg` es un agregado que une textos: junta los valores de todo el grupo en uno solo, con el separador que le digas. Muy útil para ver de un vistazo qué hay dentro de cada grupo.

El mismo bar dado de alta dos veces, con el nombre escrito distinto. Pasa constantemente en las empresas: un comercial da de alta a un cliente que ya existía porque no lo encuentra al buscarlo por el nombre. Lo resolverás en el ticket MJD-205.

9.4 Textos que significan lo mismo

Las quince provincias de 5.2 se arreglan con un `CASE` sobre la versión normalizada del texto:

SQL · EN DUCKDB

```
SELECT CASE
           WHEN lower(trim(provincia)) IN ('cantabria', 'santander')            THEN 'Cantabria'
           WHEN lower(trim(provincia)) IN ('asturias', 'principado de asturias') THEN 'Asturias'
           WHEN lower(trim(provincia)) IN ('bizkaia', 'vizcaya')                THEN 'Bizkaia'
           WHEN lower(trim(provincia)) = 'burgos'                               THEN 'Burgos'
           WHEN lower(trim(provincia)) = 'palencia'                             THEN 'Palencia'
           ELSE 'REVISAR'
       END AS provincia_limpia,
       count(*) AS clientes
FROM bronze.clientes
GROUP BY provincia_limpia
ORDER BY clientes DESC;
```

RESULTADO

provincia_limpia varchar	clientes int64
Cantabria	709
Asturias	197
Bizkaia	148
Burgos	96
Palencia	62

Dos detalles de oficio:

1. **Primero normalizar, después comparar.** `lower(trim(...))` reduce `' Cantabria'`, `'CANTABRIA'` y `'cantabria'` a una sola forma antes de compararla. Así el `CASE` tiene cinco líneas y no doce.
2. **El `ELSE 'REVISAR'`.** Si mañana aparece una provincia nueva o una forma nueva de escribirla, no se pierde ni se mete a la fuerza en otra: sale marcada para que alguien la mire. Nunca termines un `CASE` de limpieza con un `ELSE` que asigne un valor válido.

> **SANTANDER NO ES UNA PROVINCIA**
>
> Algunos clientes tienen `Santander` en el campo provincia. Es un error de quien los dio de alta (Santander es la capital, no la provincia), y lo tratamos como Cantabria. Pero eso es una **decisión**, no un hecho. Las decisiones de limpieza se apuntan y se comunican, porque alguien podría tomar otra distinta.

9.5 Códigos postales y teléfonos

Los códigos postales se cargaron como número, y un número no tiene ceros a la izquierda:

SQL · EN DUCKDB

```
SELECT municipio, codigo_postal, lpad(codigo_postal::VARCHAR, 5, '0') AS cp_correcto
FROM bronze.clientes
WHERE codigo_postal < 10000
LIMIT 4;
```

RESULTADO

```
┌──────────────────┬──────────────┬─────────────┐
│    municipio     │ codigo_postal │ cp_correcto │
│     varchar      │     int64     │   varchar   │
├──────────────────┼──────────────┼─────────────┤
│ Burgos           │          9006 │ 09006       │
│ Aranda de Duero  │          9400 │ 09400       │
│ Burgos           │          9007 │ 09007       │
│ Burgos           │          9006 │ 09006       │
└──────────────────┴──────────────┴─────────────┘
```

Burgos (09) se ha quedado con códigos de cuatro cifras. No es un fallo de DuckDB: el fichero ya llegó así, porque alguien lo abrió con Excel, que convierte en número todo lo que lo parece. La regla: **un código no es un número**, aunque esté hecho de cifras. Los códigos postales, teléfonos, NIF o números de cuenta se guardan como texto. Nunca vas a sumarlos.

Los teléfonos llegan en cuatro formatos. `regexp_replace` quita todo lo que no sea una cifra (`[^0-9]` significa "cualquier carácter que no sea del 0 al 9"; la `'g'` del final, "todos los que encuentres") y `right(..., 9)` se queda con los nueve últimos dígitos, que descarta el prefijo internacional:

SQL · EN DUCKDB

```
SELECT telefono,
       right(regexp_replace(telefono, '[^0-9]', '', 'g'), 9) AS telefono_limpio
FROM bronze.clientes
WHERE telefono LIKE '+%' OR telefono LIKE '% %' OR telefono LIKE '00%'
LIMIT 5;
```

RESULTADO

telefono varchar	telefono_limpio varchar
619 514 429	619514429
+34 942499474	942499474
979 830 090	979830090
944 933 462	944933462
947 541 825	947541825

9.6 Fechas imposibles

El `SUMMARIZE` de 9.1 avisaba del 1900. Busca todas las entregas anteriores a su pedido:

SQL · EN DUCKDB

```
SELECT CASE WHEN fecha_entrega = DATE '1900-01-01' THEN 'Fecha de relleno (1900)'
            ELSE 'Año mal tecleado' END AS tipo,
       count(*) AS pedidos
FROM bronze.pedidos
WHERE fecha_entrega < CAST(fecha_pedido AS DATE)
GROUP BY tipo;
```

RESULTADO

tipo varchar	pedidos int64
Año mal tecleado	14
Fecha de relleno (1900)	9

SQL · EN DUCKDB

```
SELECT pedido_id, fecha_pedido::DATE AS pedido, fecha_entrega AS entrega
FROM bronze.pedidos
WHERE fecha_entrega < CAST(fecha_pedido AS DATE)
  AND fecha_entrega <> DATE '1900-01-01'
ORDER BY pedido_id
LIMIT 5;
```

RESULTADO

pedido_id int64	pedido date	entrega date
2024010727	2024-06-28	2023-07-01
2024021719	2024-11-29	2023-12-02
2024022096	2024-12-04	2023-12-05
2024022140	2024-12-05	2023-12-09
2025013364	2025-07-14	2024-07-15

Los "años mal tecleados" son evidentes: pedido el 28 de junio de 2024, entregado el 1 de julio de 2023. Alguien escribió el año anterior. Es tentador corregirlos (sumar un año), pero ¿y si no fue eso? Lo prudente en la capa silver es dejar la fecha vacía y marcar el pedido, y avisar a logística de los 23 casos para que los corrijan en el ERP, que es donde de verdad hay que arreglarlo.

9.7 Huérfanos

Ya encontraste los pedidos sin cliente en 8.4. El mismo anti-join para las otras dos relaciones:

SQL · EN DUCKDB

```
SELECT 'Líneas sin pedido' AS problema, count(*) AS filas
FROM bronze.lineas_pedido AS l
LEFT JOIN bronze.pedidos AS p ON p.pedido_id = l.pedido_id
WHERE p.pedido_id IS NULL
UNION ALL
SELECT 'Líneas con producto inexistente', count(*)
FROM bronze.lineas_pedido AS l
LEFT JOIN bronze.productos AS pr ON pr.producto_id = l.producto_id
WHERE pr.producto_id IS NULL;
```

RESULTADO

```
┌──────────────────────────────────┬─────────┐
│            problema              │ filas   │
│             varchar              │ int64   │
├──────────────────────────────────┼─────────┤
│ Líneas sin pedido                │      36 │
│ Líneas con producto inexistente  │       5 │
└──────────────────────────────────┴─────────┘
```

Y no tienen el mismo tratamiento:

HUÉRFANO	QUÉ ES	QUÉ HACER
Pedido sin cliente	Una venta real de un cliente borrado	Conservarlo, con el cliente como "desconocido". La venta existió.
Línea sin pedido	Restos de un pedido que se borró	Excluirla. Sin cabecera no hay fecha, cliente ni estado: no se puede usar.
Línea con producto inexistente	Una venta real de un artículo que no está en el maestro	Conservarla, con el producto "desconocido", y avisar.

9.8 Lo raro que no es un error

Parte del criterio es saber cuándo algo raro es correcto. Hay líneas con cantidad negativa:

SQL · EN DUCKDB

```
SELECT p.estado, count(*) AS lineas_negativas
FROM bronze.lineas_pedido AS l
JOIN bronze.pedidos AS p ON p.pedido_id = l.pedido_id
WHERE l.cantidad < 0
GROUP BY p.estado;
```

RESULTADO

```
┌──────────────────┬──────────────────┐
│      estado      │ lineas_negativas │
│     varchar      │      int64       │
├──────────────────┼──────────────────┤
│ Devuelto parcial │             1166 │
└──────────────────┴──────────────────┘
```

Todas son de pedidos en estado "Devuelto parcial". La documentación del ERP (Volumen 01, apéndice A) lo dice: las devoluciones se registran como líneas en negativo. No son errores: son abonos, y tienen que restar en la facturación. Si los hubieras "limpiado", las ventas saldrían infladas.

IDEA CLAVE

Antes de borrar o corregir algo raro, pregunta (a la documentación, o a una persona) si es un error o una regla del negocio que no conocías. Y nunca borres nada en bronze: las correcciones se hacen en silver, donde siempre puedes volver atrás.

TICKETS

Una semana en Marejada

Nueve encargos de cinco departamentos. Intenta cada uno antes de mirar la solución, y guarda cada consulta en sql/02-sql-i/.

Tickets · Volumen 02

Soluciones · Volumen 02

Tickets · Volumen 02

La fecha de corte de todos los tickets es el **30 de septiembre de 2026**, el último día con datos. Cuando un ticket diga "hoy" o "los últimos tres meses", cuenta desde ahí.

TICKET MJD-201 De: Íñigo Sainz · Director comercial

Prioridad: alta Plazo: hoy

Álvaro, para la reunión de mañana necesito saber cuántos clientes activos tenemos en cada provincia.

CRITERIOS DE ACEPTACIÓN

- Una fila por provincia, con nombres correctos.
- El total tiene que ser creíble.

PISTA

"Clientes activos" puede significar dos cosas distintas en estos datos. Averigua cuáles, calcula las dos y explica la diferencia en una frase.

TICKET MJD-202 De: Íñigo Sainz · Director comercial

Prioridad: media

Quiero el top 10 de productos por facturación de 2025, con su formato. Lo voy a usar para negociar con los proveedores.

CRITERIOS DE ACEPTACIÓN

- Facturación neta de descuentos, sin pedidos cancelados y con las devoluciones restadas.
- Sin ningún dato duplicado.

TICKET MJD-203 De: Raquel Fernández · Jefa de ventas Cantabria

Prioridad: alta Plazo: mañana a primera hora

Álvaro, ¿me puedes sacar los clientes de Cantabria que llevan tres meses sin pedirnos nada? Quiero que el equipo los llame mañana.

CRITERIOS DE ACEPTACIÓN

- Un cliente por fila, con su comercial, su teléfono y la fecha de su último pedido.
- Ordenados de más a menos facturación del último año.

ENTREGABLE

La consulta en `sql/02-sql-i/` y el resultado en Excel: `entregables/mjd-203_clientes_dormidos.xlsx`.

TICKET MJD-204 De: Lucía Gómez · Administración y finanzas

Prioridad: alta

Buenos días. Íñigo dice en el comité que en marzo de 2025 vendimos 1,10 millones, y en contabilidad tenemos facturados 1,08. Son 22.000 euros de diferencia y alguien está mal. ¿Lo puedes mirar? Te dejo nuestra hoja en `datos/contabilidad/`.

CRITERIOS DE ACEPTACIÓN

- Explicar de dónde sale cada euro de diferencia.
- Comprobar provincia por provincia que, con los criterios de contabilidad, la cifra cuadra con su hoja.

TICKET MJD-205 De: Elena Pérez · Responsable de datos

Prioridad: media

En el 9.3 has visto que hay clientes dados de alta dos veces. Necesito la lista completa para pasársela a administración y que los fusionen en el ERP.

CRITERIOS DE ACEPTACIÓN

- Cada par de fichas duplicadas, con sus datos y cuántos pedidos tiene cada una.
- Una propuesta de cuál conservar de cada par, con el criterio explicado.

TICKET MJD-206 De: Marcos Diego · Logística y almacén

Prioridad: media

¿Cuánto tardamos de media en entregar desde que entra el pedido? Por almacén y por mes, este año. Me lo pide la dirección porque un cliente de Asturias se ha quejado.

TICKET MJD-207 De: Íñigo Sainz · Director comercial

Prioridad: media

Necesito las ventas de 2025 de cada comercial, por trimestre, para las evaluaciones anuales. Con su nombre, no con el número.

CRITERIOS DE ACEPTACIÓN

- Una fila por comercial: facturación de cada trimestre y del año, pedidos y ticket medio.

TICKET MJD-208 De: Paula Herrero · Marketing y tienda online

Prioridad: baja

Hola Álvaro. Lanzamos la tienda online en abril de 2025 y necesito saber cuánto pesa ya en la facturación, mes a mes, para el informe de marketing.

TICKET MJD-209 De: Marcos Diego · Logística y almacén

Prioridad: media

Necesito liberar espacio. ¿Qué tenemos en el almacén que no se haya vendido en los últimos 90 días, y cuánto dinero tenemos ahí parado?

Soluciones · Volumen 02

MJD-201 · Clientes activos por provincia

La primera definición de "activo" es la del maestro: la columna `activo`. Con la provincia normalizada del 9.4:

SQL · EN DUCKDB

```
SELECT CASE
           WHEN lower(trim(provincia)) IN ('cantabria', 'santander')            THEN 'Cantabria'
           WHEN lower(trim(provincia)) IN ('asturias', 'principado de asturias') THEN 'Asturias'
           WHEN lower(trim(provincia)) IN ('bizkaia', 'vizcaya')                THEN 'Bizkaia'
           WHEN lower(trim(provincia)) = 'burgos'                               THEN 'Burgos'
           WHEN lower(trim(provincia)) = 'palencia'                             THEN 'Palencia'
           ELSE 'REVISAR'
       END AS provincia_limpia,
       count(*) AS marcados_activos
FROM bronze.clientes
WHERE activo = 'S'
GROUP BY provincia_limpia
ORDER BY marcados_activos DESC;
```

RESULTADO

```
┌──────────────────┬──────────────────┐
│ provincia_limpia │ marcados_activos │
│     varchar      │      int64       │
├──────────────────┼──────────────────┤
│ Cantabria        │              663 │
│ Asturias         │              176 │
│ Bizkaia          │              142 │
│ Burgos           │               84 │
│ Palencia         │               60 │
└──────────────────┴──────────────────┘
```

UN ALIAS CON NOMBRE DE COLUMNA

Fíjate en que el alias es `provincia_limpia` y no `provincia`. Si lo llamas `provincia`, igual que la columna original, el `GROUP BY provincia` agrupa por la columna **original**, no por tu `CASE`, y vuelven a salir las quince provincias mal escritas, con el nombre limpio repetido en varias filas. No da ningún error. Regla: a una columna calculada, nunca le pongas el nombre de una columna que ya existe en la tabla.

La segunda definición, la que de verdad le importa a un director comercial, es "ha comprado recientemente". Con una ventana de 90 días desde la fecha de corte, y uniendo con la fecha del último pedido no cancelado de cada cliente:

SQL · EN DUCKDB

```
WITH ultima_compra AS (
    SELECT cliente_id, max(fecha_pedido) AS ultimo_pedido
    FROM bronze.pedidos
    WHERE estado <> 'Cancelado'
    GROUP BY cliente_id
)
SELECT CASE
           WHEN lower(trim(c.provincia)) IN ('cantabria', 'santander')           THEN 'Cantabria'
           WHEN lower(trim(c.provincia)) IN ('asturias', 'principado de asturias') THEN 'Asturias'
           WHEN lower(trim(c.provincia)) IN ('bizkaia', 'vizcaya')               THEN 'Bizkaia'
           WHEN lower(trim(c.provincia)) = 'burgos'                              THEN 'Burgos'
           WHEN lower(trim(c.provincia)) = 'palencia'                            THEN 'Palencia'
           ELSE 'REVISAR'
       END AS provincia_limpia,
       count(*) FILTER (WHERE c.activo = 'S') AS marcados_activos,
       count(*) FILTER (WHERE u.ultimo_pedido >= DATE '2026-09-30' - INTERVAL 90 DAY) AS
compran_ult_90_dias,
       count(*) FILTER (WHERE c.activo = 'S'
                        AND (u.ultimo_pedido IS NULL
                             OR u.ultimo_pedido < DATE '2026-09-30' - INTERVAL 90 DAY)) AS
marcados_pero_sin_compras
FROM bronze.clientes AS c
LEFT JOIN ultima_compra AS u ON u.cliente_id = c.cliente_id
GROUP BY provincia_limpia
ORDER BY marcados_activos DESC;
```

RESULTADO

provincia_limpia	marcados_activos	compran_ult_90_dias	marcados_pero_sin_compras
varchar	int64	int64	int64
Cantabria	663	620	43
Asturias	176	166	10
Bizkaia	142	135	7
Burgos	84	79	5
Palencia	60	55	5

Lo que le contestas a Íñigo, en dos frases: "En el ERP hay 1.125 clientes marcados como activos, pero de ellos 70 no han comprado nada en los últimos tres meses: o se han ido o están dormidos. Clientes que han comprado en los últimos 90 días, 1.055; te recomiendo usar esta cifra." La tabla, debajo.

DESMONTANDO LA CONSULTA

`count(*) FILTER (WHERE u.ultimo_pedido >= DATE '2026-09-30' - INTERVAL 90 DAY)`

`DATE '2026-09-30'`	La fecha de corte, escrita como fecha (no como texto) para que se pueda operar con ella.
`- INTERVAL 90 DAY`	Menos 90 días: el 2 de julio de 2026.
`u.ultimo_pedido >= ...`	El cliente compró en esa ventana.
`LEFT JOIN ultima_compra`	Para no perder a los clientes que no han comprado nunca: con `INNER JOIN` desaparecerían, y justo esos son los que importan en la última columna.

REVISIÓN DE ELENA

La clave del ticket no era el SQL, era darte cuenta de que la pregunta tenía dos respuestas. La bandera `activo` del ERP la actualiza alguien a mano cuando se acuerda, y por eso hay 70 clientes "activos" que no compran. Si le hubieras dado a Íñigo solo la primera cifra, mañana en la reunión estaría hablando de clientes que ya no existen.

Una cosa más que no te pedía el ticket pero que conviene saber antes de que te lo pregunten: estas cifras cuentan **fichas** de cliente, no clientes. Como el maestro tiene duplicados (9.3), algunos clientes cuentan dos veces. Es una diferencia pequeña, la resolverás en el MJD-205, y por ahora basta con una nota al pie.

MJD-202 · Top 10 de productos de 2025

Juntas lo que has visto: líneas sin duplicar (9.2), unidas con los pedidos para filtrar por año y estado, y con los productos para el nombre:

SQL · EN DUCKDB

```
WITH lineas AS (
    SELECT DISTINCT pedido_id, num_linea, producto_id, cantidad, precio_unitario, descuento_pct
    FROM bronze.lineas_pedido
)
SELECT pr.sku,
       pr.nombre,
       pr.formato,
       round(sum(l.cantidad * l.precio_unitario * (1 - l.descuento_pct / 100)), 2) AS facturacion_2025
FROM lineas AS l
JOIN bronze.pedidos   AS p  ON p.pedido_id    = l.pedido_id
JOIN bronze.productos AS pr ON pr.producto_id = l.producto_id
WHERE p.estado <> 'Cancelado'
  AND p.fecha_pedido >= '2025-01-01'
  AND p.fecha_pedido <  '2026-01-01'
GROUP BY pr.sku, pr.nombre, pr.formato
ORDER BY facturacion_2025 DESC
LIMIT 10;
```

RESULTADO

sku varchar	nombre varchar	formato varchar	facturacion_2025 double
ALI-0001	Anchoas de Santoña en aceite	Bandeja 12 latas	1065599.02
VIN-0025	Bodegas Valle del Nansa Blanco I.G.P. Costa …	Caja 6x75 cl	917337.46
VIN-0023	Bodegas Valle del Nansa Crianza D.O. Ribera …	Caja 6x75 cl	586205.99
VIN-0015	Hacienda Ruesga Reserva D.O.Ca. Rioja	Caja 6x75 cl	426057.45
REF-0003	Cola Bahía	Bag-in-box 18 L	421423.14
DES-0001	Vasos de cartón 25 cl	Caja 1000 uds	385835.42
CER-0066	Ola Fría Sin alcohol	Caja 24x33 cl lata	355647.02
CER-0020	Brisa Cantábrica Lager	Barril 30 L	349934.3
DES-0006	Envases take away cartón	Caja 300 uds	297448.77
REF-0016	Tónica Brava	Bag-in-box 18 L	291850.06

10 rows 4 columns

Las devoluciones se restan solas: son líneas con cantidad negativa dentro de los pedidos "Devuelto parcial", que el filtro `estado <> 'Cancelado'` conserva.

REVISIÓN DE ELENA

Bien resuelto. Tres cosas que miro siempre en un ranking:

- **Agrupar por el identificador, no solo por el nombre.** Tienes varios productos que se llaman igual en distinto formato ("Faro Norte Lager" en barril de 20 L y de 30 L). Si agrupas solo por `nombre`, los mezclas. Por eso el `sku` va en el `GROUP BY`.
- **Los `INNER JOIN` aquí están justificados**, y conviene saber por qué: las 5 líneas con producto inexistente quedan fuera del ranking, que es lo correcto (no puedes poner en el top un producto sin nombre), y no hay ninguna en 2025 que pueda entrar en el top 10. Si hubiera una venta enorme de un producto desconocido, tendrías que contarlo.
- **"Facturación" es una palabra peligrosa.** Íñigo habla de ventas por fecha de pedido; Lucía, en contabilidad, factura por fecha de entrega. Para un ranking anual la diferencia es pequeña, pero ponle a la columna un nombre que diga qué es. Lo vas a entender del todo en el MJD-204.

MJD-203 · Clientes dormidos de Cantabria

Un cliente "dormido" es uno que ha comprado alguna vez y cuyo último pedido es anterior al 2 de julio de 2026. Además del último pedido, Raquel quiere ordenar por la facturación del último año, que se calcula en otra consulta y se une:

SQL · EN DUCKDB

```
WITH lineas AS (
    SELECT DISTINCT pedido_id, num_linea, producto_id, cantidad, precio_unitario, descuento_pct
    FROM bronze.lineas_pedido
),
pedidos_validos AS (
    SELECT * FROM bronze.pedidos WHERE estado <> 'Cancelado'
),
ultima_compra AS (
    SELECT cliente_id, max(fecha_pedido)::DATE AS ultimo_pedido
    FROM pedidos_validos
    GROUP BY cliente_id
),
ventas_ultimo_anio AS (
    SELECT p.cliente_id,
           sum(l.cantidad * l.precio_unitario * (1 - l.descuento_pct / 100)) AS ventas
    FROM pedidos_validos AS p
    JOIN lineas AS l ON l.pedido_id = p.pedido_id
    WHERE p.fecha_pedido >= DATE '2026-09-30' - INTERVAL 1 YEAR
    GROUP BY p.cliente_id
)
SELECT c.cliente_id,
       c.nombre_comercial,
       c.municipio,
       right(regexp_replace(c.telefono, '[^0-9]', '', 'g'), 9) AS telefono,
       co.nombre || ' ' || co.apellidos AS comercial,
       u.ultimo_pedido,
       round(coalesce(v.ventas, 0), 2) AS ventas_ultimo_anio
FROM bronze.clientes AS c
JOIN ultima_compra AS u ON u.cliente_id = c.cliente_id
LEFT JOIN ventas_ultimo_anio AS v ON v.cliente_id = c.cliente_id
LEFT JOIN bronze.comerciales AS co ON co.comercial_id = c.comercial_id
WHERE lower(trim(c.provincia)) IN ('cantabria', 'santander')
  AND u.ultimo_pedido < DATE '2026-09-30' - INTERVAL 90 DAY
ORDER BY ventas_ultimo_anio DESC;
```

RESULTADO

cliente_id int64	nombre_comercial varchar	…	ultimo_pedido date	ventas_ultimo_anio double
10637	Supermercado La Fuente	…	2026-04-27	50860.3
11193	Supermercado La Fuente S.L.	…	2026-04-06	43887.78
11073	Supermercado El Faro	…	2026-03-11	31952.41
10782	Supermercado El Botxo	…	2026-07-01	26797.64
10915	Hotel El Sardinero	…	2026-04-18	17319.27
.	.	…	.	.
.	.	…	.	.
.	.	…	.	.
10537	Alimentación Malecón	…	2024-12-09	0.0
10624	Restaurante Bitácora	…	2025-08-15	0.0
10883	Bar Las Rocas Viejo	…	2024-08-02	0.0
10267	Hotel Marejadilla	…	2024-09-13	0.0
10716	Restaurante La Sidra	…	2025-06-13	0.0

86 rows (10 shown) 7 columns (4 shown)

Ochenta y seis clientes. Y para el Excel que pide Raquel, DuckDB escribe directamente un fichero Excel con `COPY`. Necesita la extensión de Excel: se instala una vez (`INSTALL`) y se carga en cada sesión (`LOAD`):

SQL

```
INSTALL excel;
LOAD excel;
COPY (
    -- aquí va la consulta completa de arriba, sin el punto y coma final
) TO 'entregables/mjd-203_clientes_dormidos.xlsx' WITH (FORMAT xlsx, HEADER true);
```

DESMONTANDO EL COMANDO

`COPY ( consulta ) TO 'fichero.xlsx' WITH (FORMAT xlsx, HEADER true)`

`COPY ( ... )`	Copia el resultado de la consulta de dentro…
`TO 'entregables/...xlsx'`	…a este fichero.
`FORMAT xlsx`	En formato Excel. Con `FORMAT csv` sería un CSV.
`HEADER true`	Con los nombres de las columnas en la primera fila.

REVISIÓN DE ELENA

Esto ya es un entregable de verdad. Te cuento lo que cambiaría yo antes de mandárselo a Raquel, que es lo que separa una consulta correcta de un trabajo útil:

- **Mira la columna `ultimo_pedido` antes de enviar.** Hay clientes cuyo último pedido es de 2024. Esos no están dormidos: se fueron hace más de un año, y llamarlos mañana tiene otra conversación. Yo añadiría una columna con los meses sin pedir, o le daría a Raquel dos pestañas: dormidos de este año y perdidos.
- **El comercial que sale es el asignado ahora**, que es el que tiene que llamar. Bien elegido.
- **Teléfono limpio:** bien. Raquel lo va a copiar al móvil.
- La mitad de estos clientes sigue con `activo = 'S'` en el ERP. Es la misma lección del MJD-201, vista desde el otro lado.

MJD-204 · Marzo de 2025 no cuadra

Este es el ticket más importante del volumen, porque es exactamente lo que pasa en las empresas: dos departamentos, dos cifras, y cada uno convencido de la suya. La forma profesional de resolverlo es un **puente**: partir de una cifra y llegar a la otra paso a paso, explicando cuánto aporta cada diferencia.

Paso 1. Reproducir la cifra de Íñigo. Todo lo pedido en marzo, tal cual:

SQL · EN DUCKDB

```
SELECT round(sum(l.cantidad * l.precio_unitario * (1 - l.descuento_pct / 100)), 2) AS cifra_comercial
FROM bronze.pedidos AS p
JOIN bronze.lineas_pedido AS l ON l.pedido_id = p.pedido_id
WHERE p.fecha_pedido >= '2025-03-01' AND p.fecha_pedido < '2025-04-01';
```

RESULTADO

```
┌─────────────────┐
│ cifra_comercial │
│     double      │
├─────────────────┤
│      1104552.29 │
└─────────────────┘
```

1,10 millones. Es la cifra de Íñigo.

Paso 2. Leer la hoja de contabilidad. DuckDB lee Excel con `read_xlsx`. Primer intento:

SQL · EN DUCKDB

```
SELECT * FROM read_xlsx('datos/contabilidad/facturacion_mensual_2024-2026.xlsx') LIMIT 3;
```

RESULTADO

```
┌──────────────────────────────────────────────────────────────────────────────┐
│               MAREJADA DISTRIBUCIÓN, S.L. — Departamento de Administración   │
│                                    varchar                                   │
├──────────────────────────────────────────────────────────────────────────────┤
│ Base imponible facturada por mes de entrega y provincia (sin IVA). Generado el 05/10/2026. │
└──────────────────────────────────────────────────────────────────────────────┘
```

Ha leído el título de la hoja como si fuera la cabecera. Abre el Excel y lo verás: dos filas de título, una vacía, y la tabla empieza en la fila 4. Se le dice dónde está la tabla con `range`, en notación de Excel:

SQL · EN DUCKDB

```
SELECT *
FROM read_xlsx('datos/contabilidad/facturacion_mensual_2024-2026.xlsx', range = 'A4:E1000')
WHERE "Año" = 2025 AND "Mes" = 'marzo';
```

RESULTADO

```
┌──────────┬──────────┬──────────┬──────────────────┬──────────────────────────┐
│   Año    │   Mes    │ Provincia │ Base imponible (€) │ Gastos de envío incluidos (€) │
│  double  │ varchar  │  varchar  │      double      │          double          │
├──────────┼──────────┼──────────┼──────────────────┼──────────────────────────┤
│   2025.0 │ marzo    │ Asturias  │        205455.25 │                   1080.0 │
│   2025.0 │ marzo    │ Bizkaia   │          9992.87 │                     36.0 │
│   2025.0 │ marzo    │ Burgos    │        116563.83 │                    504.0 │
│   2025.0 │ marzo    │ Cantabria │        684893.81 │                   2868.0 │
│   2025.0 │ marzo    │ Palencia  │         65337.74 │                    450.0 │
└──────────┴──────────┴──────────┴──────────────────┴──────────────────────────┘
```

Las columnas con espacios, tildes o símbolos van entre comillas dobles: `"Año"`, `"Base imponible (€)"`. La hoja de Notas del Excel explica sus criterios: se factura **en el mes de entrega**, los cancelados no se facturan, las devoluciones se restan y la base **incluye los gastos de envío**.

Paso 3. El puente. Cada criterio de contabilidad, aplicado de uno en uno:

SQL · EN DUCKDB

```
WITH lineas_unicas AS (
    SELECT DISTINCT pedido_id, num_linea, producto_id, cantidad, precio_unitario, descuento_pct
    FROM bronze.lineas_pedido
),
importe_todas AS (
    SELECT pedido_id, sum(cantidad * precio_unitario * (1 - descuento_pct / 100)) AS importe
    FROM bronze.lineas_pedido GROUP BY pedido_id
),
importe_unicas AS (
    SELECT pedido_id, sum(cantidad * precio_unitario * (1 - descuento_pct / 100)) AS importe
    FROM lineas_unicas GROUP BY pedido_id
)
SELECT '1. Pedido en marzo, todo' AS paso,
       round(sum(i.importe), 2) AS importe
FROM bronze.pedidos p JOIN importe_todas i USING (pedido_id)
WHERE p.fecha_pedido >= '2025-03-01' AND p.fecha_pedido < '2025-04-01'
UNION ALL
SELECT '2. Sin cancelados',
       round(sum(i.importe), 2)
FROM bronze.pedidos p JOIN importe_todas i USING (pedido_id)
WHERE p.fecha_pedido >= '2025-03-01' AND p.fecha_pedido < '2025-04-01'
  AND p.estado <> 'Cancelado'
UNION ALL
SELECT '3. Sin líneas duplicadas',
       round(sum(i.importe), 2)
FROM bronze.pedidos p JOIN importe_unicas i USING (pedido_id)
WHERE p.fecha_pedido >= '2025-03-01' AND p.fecha_pedido < '2025-04-01'
  AND p.estado <> 'Cancelado'
UNION ALL
SELECT '4. Por mes de entrega, no de pedido',
       round(sum(i.importe), 2)
FROM bronze.pedidos p JOIN importe_unicas i USING (pedido_id)
WHERE p.fecha_entrega >= '2025-03-01' AND p.fecha_entrega < '2025-04-01'
  AND p.estado IN ('Entregado', 'Devuelto parcial')
UNION ALL
SELECT '5. Más gastos de envío',
       round(sum(i.importe + p.gastos_envio), 2)
FROM bronze.pedidos p JOIN importe_unicas i USING (pedido_id)
WHERE p.fecha_entrega >= '2025-03-01' AND p.fecha_entrega < '2025-04-01'
  AND p.estado IN ('Entregado', 'Devuelto parcial');
```

RESULTADO

```
┌──────────────────────────────────────┬──────────┐
│                paso                  │ importe  │
│              varchar                 │  double  │
├──────────────────────────────────────┼──────────┤
│ 1. Pedido en marzo, todo             │ 1104552.29 │
│ 2. Sin cancelados                    │ 1066085.78 │
│ 3. Sin líneas duplicadas             │ 1063694.42 │
│ 4. Por mes de entrega, no de pedido  │  1077305.5 │
│ 5. Más gastos de envío               │  1082243.5 │
└──────────────────────────────────────┴──────────┘
```

Fíjate en cómo están sumados los gastos de envío en el paso 5: el importe de las líneas ya está agregado por pedido (`importe_unicas`), así que cada pedido es una fila y los gastos no se multiplican. Es la trampa del abanico del 8.6, evitada.

Lo que explica cada paso, con las diferencias:

PASO	QUÉ CAMBIA	POR QUÉ
1 → 2	Fuera los cancelados	Un pedido cancelado no se factura.

PASO	QUÉ CAMBIA	POR QUÉ
2 → 3	Fuera las líneas duplicadas	Un fallo de sincronización del ERP las insertó dos veces. Esa diferencia es un **error**, no un criterio: afecta también a la cifra de Íñigo.
3 → 4	Mes de entrega en vez de mes de pedido	Lo pedido el 31 de marzo y entregado el 2 de abril es venta de marzo para Íñigo y factura de abril para Lucía. Y al revés con lo pedido a final de febrero.
4 → 5	Se suman los portes	La factura incluye los gastos de envío; las ventas de producto, no.

Paso 4. Comprobar provincia por provincia. El puente demuestra que el total cuadra. Lucía pidió además comprobarlo por provincia:

SQL · EN DUCKDB

```
WITH lineas_unicas AS (
    SELECT DISTINCT pedido_id, num_linea, producto_id, cantidad, precio_unitario, descuento_pct
    FROM bronze.lineas_pedido
),
importe AS (
    SELECT pedido_id, sum(cantidad * precio_unitario * (1 - descuento_pct / 100)) AS importe
    FROM lineas_unicas GROUP BY pedido_id
),
nuestro AS (
    SELECT CASE
               WHEN lower(trim(c.provincia)) IN ('cantabria', 'santander')            THEN 'Cantabria'
               WHEN lower(trim(c.provincia)) IN ('asturias', 'principado de asturias') THEN 'Asturias'
               WHEN lower(trim(c.provincia)) IN ('bizkaia', 'vizcaya')                THEN 'Bizkaia'
               WHEN lower(trim(c.provincia)) = 'burgos'                               THEN 'Burgos'
               WHEN lower(trim(c.provincia)) = 'palencia'                             THEN 'Palencia'
               ELSE 'Sin asignar'
           END AS provincia,
           round(sum(i.importe + p.gastos_envio), 2) AS calculado
    FROM bronze.pedidos AS p
    JOIN importe AS i ON i.pedido_id = p.pedido_id
    LEFT JOIN bronze.clientes AS c ON c.cliente_id = p.cliente_id
    WHERE p.fecha_entrega >= '2025-03-01' AND p.fecha_entrega < '2025-04-01'
      AND p.estado IN ('Entregado', 'Devuelto parcial')
    GROUP BY 1
),
contabilidad AS (
    SELECT "Provincia" AS provincia, "Base imponible (€)" AS contabilidad
    FROM read_xlsx('datos/contabilidad/facturacion_mensual_2024-2026.xlsx', range = 'A4:E1000')
    WHERE "Año" = 2025 AND "Mes" = 'marzo'
)
SELECT provincia, n.calculado, k.contabilidad,
       round(n.calculado - k.contabilidad, 2) AS diferencia
FROM nuestro AS n
FULL JOIN contabilidad AS k USING (provincia)
ORDER BY provincia;
```

RESULTADO

```
┌────────────┬────────────┬──────────────┬────────────┐
│  provincia │ calculado  │ contabilidad │ diferencia │
│   varchar  │   double   │    double    │   double   │
├────────────┼────────────┼──────────────┼────────────┤
│ Asturias   │ 205455.25  │    205455.25 │        0.0 │
│ Bizkaia    │   9992.87  │      9992.87 │        0.0 │
│ Burgos     │ 116563.83  │    116563.83 │        0.0 │
│ Cantabria  │ 678588.45  │    684893.81 │   -6305.36 │
│ Palencia   │  65337.74  │     65337.74 │        0.0 │
│ Sin asignar│   6305.36  │         NULL │       NULL │
└────────────┴────────────┴──────────────┴────────────┘
```

Cuadran cuatro provincias al céntimo. Y en Cantabria pasa algo muy interesante: a nosotros nos faltan 6.305,36 €, y en una fila que contabilidad no tiene, `Sin asignar`, nos sobran exactamente 6.305,36 €. Son los pedidos de los clientes borrados del maestro (8.4): nosotros no sabemos de qué provincia eran, pero contabilidad sí, porque los tiene en su programa de facturación. Eran clientes de Cantabria.

Eso lo ha encontrado el `FULL JOIN`, y por eso estaba ahí: con un `INNER JOIN`, la fila `Sin asignar` habría desaparecido y te habrías quedado con una diferencia en Cantabria sin explicación.

La respuesta a Lucía (y a Íñigo, en copia): **las dos cifras están bien calculadas, pero miden cosas distintas**: comercial cuenta lo pedido en el mes y contabilidad lo entregado, con portes. Y por el camino han salido dos errores que afectan a los informes comerciales: incluyen los pedidos cancelados (38.000 € en marzo) y unos 2.400 € de líneas duplicadas por un fallo de sincronización del ERP. Con los criterios de contabilidad, el total cuadra al céntimo, y por provincias también, una vez asignados a Cantabria los seis clientes borrados.

REVISIÓN DE ELENA

Esto es exactamente lo que hay que hacer, y te diré por qué este ticket vale más que los otros ocho juntos. Una discrepancia entre departamentos no se resuelve diciendo quién tiene razón: se resuelve con un puente que todo el mundo pueda seguir. Así nadie queda mal, y de paso has encontrado dos errores reales (cancelados y duplicados) que afectaban a todos los informes comerciales. Y pídele a Lucía la provincia de los seis clientes borrados: así la capa silver los podrá asignar bien.

Ojo con una cosa que no se ve en marzo pero existe. Contabilidad usa la fecha de entrega tal como está en el ERP, y en el 9.6 viste que hay 14 fechas con el año mal tecleado. Esos pedidos están facturados en el mes equivocado: un pedido de julio de 2026 aparece facturado en julio de 2025. No es tu error, pero se lo tienes que contar a Lucía.

MJD-205 · Clientes duplicados

Se identifican los NIF repetidos (9.3) y, para cada ficha, se cuentan sus pedidos:

SQL · EN DUCKDB

```
WITH fichas AS (
    SELECT *, upper(replace(trim(nif), '-', '')) AS nif_norm
    FROM bronze.clientes
),
nif_duplicados AS (
    SELECT nif_norm FROM fichas GROUP BY nif_norm HAVING count(*) > 1
),
pedidos_por_cliente AS (
    SELECT cliente_id, count(*) AS pedidos, min(fecha_pedido)::DATE AS primer_pedido
    FROM bronze.pedidos GROUP BY cliente_id
)
SELECT f.nif_norm, f.cliente_id, f.nombre_comercial, f.municipio, f.fecha_alta,
       coalesce(pc.pedidos, 0) AS pedidos
FROM fichas AS f
JOIN nif_duplicados USING (nif_norm)
LEFT JOIN pedidos_por_cliente AS pc ON pc.cliente_id = f.cliente_id
ORDER BY f.nif_norm, f.cliente_id;
```

RESULTADO

nif_norm varchar	cliente_id int64	nombre_comercial varchar	municipio varchar	fecha_alta date	pedidos int64
A40069065	10135	Restaurante La Gaviota	Torrelavega	2016-10-16	40
A40069065	11187	Restaurante La Gaviot…	Torrelavega	2024-08-30	26
A45081792	10768	Bar El Rincón	San Vicente de la Barq…	2016-08-04	40
A45081792	11183	Bar-Restaurante El Ri…	San Vicente de la Barq…	2024-06-23	20
A48106850	11150	Bar Arena Centro	Santander	2020-08-03	5
A48106850	11185	Bar Arena Centro	Santander	2025-01-26	0
.	.	.	.	.	.
.	.	.	.	.	.
.	.	.	.	.	.
J42238871	10409	Restaurante La Concha	Santander	2023-01-01	74
J42238871	11195	Restaurante La Concha…	Santander	2024-08-24	41
J44083998	10642	Restaurante El Espolón	Aranda de Duero	2017-07-18	91
J44083998	11196	Restaurante El Espolón	Aranda de Duero	2025-01-23	34
J45802421	10859	Alimentación Velero	San Vicente de la Barq…	2026-06-16	6
J45802421	11205	Alimentación Velero S…	San Vicente de la Barq…	2026-09-20	0

74 rows (12 shown) 6 columns

Y el resumen:

SQL · EN DUCKDB

```
WITH fichas AS (
    SELECT *, upper(replace(trim(nif), '-', '')) AS nif_norm FROM bronze.clientes
),
dup AS (
    SELECT nif_norm, min(cliente_id) AS ficha_a_conservar
    FROM fichas GROUP BY nif_norm HAVING count(*) > 1
),
fichas_sobrantes AS (
    SELECT f.cliente_id
    FROM fichas AS f
    JOIN dup AS d ON d.nif_norm = f.nif_norm
    WHERE f.cliente_id <> d.ficha_a_conservar
)
SELECT (SELECT count(*) FROM dup)              AS nif_duplicados,
       (SELECT count(*) FROM fichas_sobrantes) AS fichas_sobrantes,
       (SELECT count(*) FROM bronze.pedidos
        WHERE cliente_id IN (SELECT cliente_id FROM fichas_sobrantes)) AS pedidos_a_traspasar;
```

RESULTADO

nif_duplicados	fichas_sobrantes	pedidos_a_traspasar
int64	int64	int64
37	37	1007

Cada `(SELECT ...)` entre paréntesis dentro del `SELECT` devuelve un solo número: es una **subconsulta escalar**. Es la forma de poner en una misma fila cifras que salen de consultas distintas.

La propuesta: **conservar la ficha más antigua** de cada par (el `cliente_id` más bajo, que es la primera que se dio de alta) y pasarle los pedidos de la otra. El criterio es defendible porque la ficha antigua es la que tiene el historial completo, y las nuevas se crearon por error cuando el cliente ya existía.

REVISIÓN DE ELENA

Bien, y fíjate en lo que **no** has hecho, que es lo más importante: no has tocado los datos. La solución de verdad no es borrar fichas en tu base de datos, sino que administración las fusione en el ERP, que es el origen. Si las "arreglas" solo en tu lado, el mes que viene llegarán otra vez duplicadas en el siguiente CSV.

Mientras tanto, en la capa silver harás lo prudente: una columna que diga, para cada ficha, cuál es su ficha maestra. Así los análisis agrupan bien y bronze sigue intacto.

MJD-206 · Plazo de entrega

La diferencia entre la fecha de entrega y la de pedido, en días, por almacén y mes de 2026:

SQL · EN DUCKDB

```
SELECT almacen,
       date_trunc('month', fecha_pedido)::DATE AS mes,
       count(*) AS pedidos,
       round(avg(fecha_entrega - fecha_pedido::DATE), 2) AS dias_medios,
       round(100.0 * count(*) FILTER (WHERE fecha_entrega - fecha_pedido::DATE <= 1) / count(*), 1) AS
pct_en_24h
FROM bronze.pedidos
WHERE estado IN ('Entregado', 'Devuelto parcial')
  AND fecha_pedido >= '2026-01-01'
  AND fecha_entrega >= fecha_pedido::DATE
GROUP BY almacen, mes
ORDER BY almacen, mes;
```

RESULTADO

almacen varchar	mes date	pedidos int64	dias_medios double	pct_en_24h double
Guarnizo	2026-01-01	1660	2.2	36.5
Guarnizo	2026-02-01	1530	2.12	37.9
Guarnizo	2026-03-01	1954	2.07	41.5
Guarnizo	2026-04-01	2144	2.18	39.1
Guarnizo	2026-05-01	2210	2.2	35.3
Guarnizo	2026-06-01	2576	2.09	40.2
Guarnizo	2026-07-01	3093	2.21	36.6
Guarnizo	2026-08-01	2850	2.06	40.4
Guarnizo	2026-09-01	2094	2.17	38.2
Llanera	2026-01-01	343	2.22	36.7
Llanera	2026-02-01	306	2.12	37.3
Llanera	2026-03-01	323	2.14	35.6
Llanera	2026-04-01	438	2.1	41.1
Llanera	2026-05-01	431	2.21	35.0
Llanera	2026-06-01	493	2.12	38.3
Llanera	2026-07-01	573	2.2	39.4
Llanera	2026-08-01	591	2.1	40.1
Llanera	2026-09-01	393	2.27	36.9

18 rows 5 columns

El filtro `fecha_entrega >= fecha_pedido::DATE` deja fuera las fechas imposibles del 9.6. ¿Importa? Esto es lo que saldría sin él, en los meses con fechas de 1900:

SQL · EN DUCKDB

```
SELECT almacen,
       date_trunc('month', fecha_pedido)::DATE AS mes,
       round(avg(fecha_entrega - fecha_pedido::DATE), 2) AS dias_medios_sin_filtrar
FROM bronze.pedidos
WHERE estado IN ('Entregado', 'Devuelto parcial')
  AND fecha_pedido >= '2026-04-01' AND fecha_pedido < '2026-05-01'
GROUP BY almacen, mes;
```

RESULTADO

almacen varchar	mes date	dias_medios_sin_filtrar double
Llanera	2026-04-01	2.1
Guarnizo	2026-04-01	-40.81

Dos pedidos con fecha de 1900, entre más de dos mil, hunden la media de abril de Guarnizo hasta casi −41 días: cada uno de ellos «aporta» unos −45.000 días. Una media es muy sensible a los valores extremos.

La respuesta a Marcos: los dos almacenes entregan de media en algo más de dos días, de forma muy estable todo el año, y Llanera no es más lento que Guarnizo. La queja del cliente de Asturias habrá que mirarla pedido a pedido.

> **REVISIÓN DE ELENA**
>
> Correcto. Dos comentarios. Uno: lo más útil para logística no suele ser la media, sino el porcentaje que cumple el objetivo (aquí lo has calculado: entregas en 24 horas). Las medias esconden los casos malos, y los clientes se quejan por los casos malos. Dos: para la queja concreta, lo siguiente sería sacar los pedidos de ese cliente con su plazo, uno por uno. Si Marcos te dice quién es, son cinco minutos.

MJD-207 · Ventas por comercial y trimestre

El comercial que cuenta es el que figura **en el pedido** (quien hizo la venta), no el asignado ahora al cliente. Una fila por comercial y los trimestres en columnas, con `FILTER`:

SQL · EN DUCKDB

```
WITH lineas AS (
    SELECT DISTINCT pedido_id, num_linea, producto_id, cantidad, precio_unitario, descuento_pct
    FROM bronze.lineas_pedido
),
ventas_pedido AS (
    SELECT p.pedido_id, p.comercial_id, quarter(p.fecha_pedido) AS trimestre,
           sum(l.cantidad * l.precio_unitario * (1 - l.descuento_pct / 100)) AS ventas
    FROM bronze.pedidos AS p
    JOIN lineas AS l ON l.pedido_id = p.pedido_id
    WHERE p.estado <> 'Cancelado'
      AND p.fecha_pedido >= '2025-01-01' AND p.fecha_pedido < '2026-01-01'
    GROUP BY p.pedido_id, p.comercial_id, trimestre
)
SELECT co.nombre || ' ' || co.apellidos AS comercial,
       round(sum(v.ventas) FILTER (WHERE trimestre = 1)) AS t1,
       round(sum(v.ventas) FILTER (WHERE trimestre = 2)) AS t2,
       round(sum(v.ventas) FILTER (WHERE trimestre = 3)) AS t3,
       round(sum(v.ventas) FILTER (WHERE trimestre = 4)) AS t4,
       round(sum(v.ventas)) AS total_2025,
       count(*) AS pedidos,
       round(sum(v.ventas) / count(*)) AS ticket_medio
FROM ventas_pedido AS v
JOIN bronze.comerciales AS co ON co.comercial_id = v.comercial_id
GROUP BY comercial
ORDER BY total_2025 DESC;
```

RESULTADO

comercial varchar	t1 double	t2 double	t3 double	…	total_2025 double	pedidos int64	ticket_medio double
Rubén Martín Calvo	463175.0	605168.0	706552.0	…	2349289.0	3851	610.0
Raquel Fernández Cobo	443809.0	563584.0	655773.0	…	2187256.0	3649	599.0
Pablo Revuelta Gómez	385197.0	542811.0	714463.0	…	2141292.0	4040	530.0
Daniel Gutiérrez Pér…	340736.0	432446.0	606866.0	…	1848675.0	3168	584.0
Adrián Suárez Fernán…	299867.0	379394.0	500170.0	…	1611782.0	2167	744.0
Nuria Álvarez Menénd…	268434.0	355033.0	420369.0	…	1398780.0	2550	549.0
Sara Ceballos Ortiz	250752.0	275255.0	351164.0	…	1208264.0	2328	519.0
Lorena Arce Díaz	158982.0	237914.0	255111.0	…	864615.0	1795	482.0
Celia Gómez Sáiz	NULL	NULL	392942.0	…	712589.0	1374	519.0
Héctor Bustamante La…	265321.0	339841.0	NULL	…	605161.0	1157	523.0
Mikel Goikoetxea Ara…	7939.0	51052.0	146274.0	…	386225.0	618	625.0
Ane Etxeberria Zubiri	2125.0	46002.0	116202.0	…	305448.0	523	584.0

12 rows 8 columns (7 shown)

Fíjate en el diseño en dos pasos: primero una fila por pedido con sus ventas (`ventas_pedido`), y después se agrega por comercial. Así `count(*)` cuenta pedidos y no líneas, y el ticket medio es por pedido. Si hubieras contado sobre las líneas, el "ticket medio" sería el importe medio de una línea, que no es lo que Íñigo entiende por ticket.

Los huecos tienen explicación: Héctor Bustamante dejó la empresa el 30 de junio (sin ventas en T3 y T4) y Celia Gómez entró en junio para sustituirle; Ane y Mikel empezaron en Bizkaia en febrero y marzo.

REVISIÓN DE ELENA

Para unas evaluaciones, esto es delicado, y por eso hay que acompañar la tabla con dos notas. La primera, que comparar comerciales por facturación total no es justo si no se tiene en cuenta la zona: quien lleva Santander tiene más clientes que quien abre Bizkaia. La segunda, que Héctor y Celia comparten cartera en 2025. No es tu trabajo decidir cómo se evalúa a nadie, pero sí que quien decida tenga el contexto.

Y un detalle técnico: el comercial del pedido y el asignado al cliente no siempre coinciden. Has usado el del pedido, que es lo correcto para medir ventas. Dilo en el correo, porque es la primera pregunta que te harán.

MJD-208 · El peso de la web

SQL · EN DUCKDB

```
WITH lineas AS (
    SELECT DISTINCT pedido_id, num_linea, producto_id, cantidad, precio_unitario, descuento_pct
    FROM bronze.lineas_pedido
),
ventas_pedido AS (
    SELECT p.pedido_id, p.canal, date_trunc('month', p.fecha_pedido)::DATE AS mes,
           sum(l.cantidad * l.precio_unitario * (1 - l.descuento_pct / 100)) AS ventas
    FROM bronze.pedidos AS p
    JOIN lineas AS l ON l.pedido_id = p.pedido_id
    WHERE p.estado <> 'Cancelado'
      AND p.fecha_pedido >= '2025-04-01'
    GROUP BY p.pedido_id, p.canal, mes
)
SELECT mes,
       round(sum(ventas)) AS ventas_totales,
       round(100 * sum(ventas) FILTER (WHERE canal = 'Web B2B')   / sum(ventas), 1) AS pct_web,
       round(100 * sum(ventas) FILTER (WHERE canal = 'Comercial') / sum(ventas), 1) AS pct_comercial,
       round(100 * sum(ventas) FILTER (WHERE canal = 'Teléfono')  / sum(ventas), 1) AS pct_telefono
FROM ventas_pedido
GROUP BY mes
ORDER BY mes;
```

RESULTADO

mes date	ventas_totales double	pct_web double	pct_comercial double	pct_telefono double
2025-04-01	1218471.0	5.4	65.0	29.6
2025-05-01	1256817.0	6.4	65.6	28.0
2025-06-01	1353213.0	8.8	64.2	27.1
2025-07-01	1718695.0	8.1	64.5	27.4
2025-08-01	1673393.0	10.8	60.8	28.5
2025-09-01	1473799.0	12.1	61.6	26.3
2025-10-01	1290332.0	13.1	60.4	26.4
2025-11-01	1120485.0	15.1	60.7	24.2
2025-12-01	1627833.0	13.7	61.7	24.6
2026-01-01	1187705.0	18.0	57.1	24.9
2026-02-01	1066618.0	16.8	58.3	24.9
2026-03-01	1351890.0	17.6	58.6	23.8
2026-04-01	1500290.0	19.3	55.5	25.2
2026-05-01	1582346.0	20.9	56.6	22.5
2026-06-01	1805026.0	21.3	54.5	24.2
2026-07-01	2142228.0	23.3	53.6	23.1
2026-08-01	1975639.0	26.1	52.6	21.3
2026-09-01	1571643.0	27.6	49.4	23.0

18 rows 5 columns

La web pasa del 5 % en abril de 2025 a casi el 28 % en septiembre de 2026. Lo que gana la web lo pierden sobre todo las visitas comerciales (del 65 % al 49 %) y, en menor medida, el teléfono. Es la historia que Paula quiere contar, y los datos la respaldan.

REVISIÓN DE ELENA

Bien. Si quieres sumar puntos con Paula, añade una frase sobre lo que le va a interesar a continuación: ¿los pedidos de la web son de clientes nuevos o de clientes de siempre que han cambiado de canal? Es otra consulta (Volumen 03), pero proponerla demuestra que entiendes para qué quiere el dato.

Y una advertencia para el futuro: estas cifras salen del ERP. La tienda online tiene su propio fichero (el JSON del MJD-100), y si los dos sistemas no coinciden, Paula y tú podéis acabar con dos cifras distintas, como Lucía e Íñigo. Lo comprobarás en el Volumen 06.

MJD-209 · Stock parado

Tres piezas: el inventario con el SKU normalizado (8.7), la fecha de la última venta de cada producto, y el coste para valorar el stock:

SQL · EN DUCKDB

```
WITH inventario AS (
    SELECT almacen, unidades, upper(replace(trim(sku), '-', '')) AS sku_norm
    FROM bronze.inventario
),
productos AS (
    SELECT producto_id, sku, nombre, formato, coste_unitario, activo,
           upper(replace(trim(sku), '-', '')) AS sku_norm
    FROM bronze.productos
),
ultima_venta AS (
    SELECT l.producto_id, max(p.fecha_pedido)::DATE AS ultima_venta
    FROM bronze.lineas_pedido AS l
    JOIN bronze.pedidos AS p ON p.pedido_id = l.pedido_id
    WHERE p.estado <> 'Cancelado'
    GROUP BY l.producto_id
)
SELECT i.almacen, pr.sku, pr.nombre, pr.formato, pr.activo,
       uv.ultima_venta,
       i.unidades,
       round(i.unidades * pr.coste_unitario, 2) AS valor_coste
FROM inventario AS i
JOIN productos AS pr ON pr.sku_norm = i.sku_norm
LEFT JOIN ultima_venta AS uv ON uv.producto_id = pr.producto_id
WHERE i.unidades > 0
  AND (uv.ultima_venta IS NULL OR uv.ultima_venta < DATE '2026-09-30' - INTERVAL 90 DAY)
ORDER BY valor_coste DESC;
```

RESULTADO

almacen varchar	sku varchar	nombre varchar	…	ultima_venta date	unidades int64	valor_coste double
Llanera	CER-0065	Ola Fría Sin alcohol	…	2025-04-30	37	1999.85
Guarnizo	DES-0002	Vasos de plástico reutilizabl…	…	2025-04-29	39	1606.02
Guarnizo	CER-0021	Brisa Cantábrica Lager	…	2024-12-10	29	1473.78
Guarnizo	CER-0065	Ola Fría Sin alcohol	…	2025-04-30	27	1459.35
Guarnizo	ALI-0018	Croissant congelado	…	2024-12-27	39	563.55
Llanera	VIN-0008	Viñas del Asón Txakoli D.O. G…	…	2025-08-21	18	537.84
.	.	.	…	.	.	.
.	.	.	…	.	.	.
.	.	.	…	.	.	.
Guarnizo	LIM-0003	Desengrasante cocina Higiene …	…	2025-01-16	9	94.95
Guarnizo	ALI-0008	Queso de nata de Cantabria	…	2024-12-31	9	84.51
Guarnizo	CER-0077	Abadía de Liébana Tostada	…	2025-02-06	5	74.45
Llanera	CER-0001	Faro Norte Lager	…	2025-10-01	1	70.33
Guarnizo	LIM-0017	Bayetas microfibra LimpiaPro	…	2024-09-09	6	30.18
Guarnizo	CAF-0013	Infusión manzanilla Café Bahía	…	2024-12-26	2	8.06

22 rows (12 shown) 8 columns (6 shown)

Y lo que hay en el almacén sin ficha en el maestro de productos:

SQL · EN DUCKDB

```
SELECT i.almacen, i.sku, i.descripcion, i.unidades
FROM bronze.inventario AS i
LEFT JOIN bronze.productos AS pr
       ON upper(replace(trim(pr.sku), '-', '')) = upper(replace(trim(i.sku), '-', ''))
WHERE pr.producto_id IS NULL
ORDER BY i.sku;
```

RESULTADO

almacen varchar	sku varchar	descripcion varchar	unidades int64
Guarnizo	OBS-0001	Artículo antiguo sin ficha 1	2
Guarnizo	OBS-0002	Artículo antiguo sin ficha 2	4
Guarnizo	OBS-0003	Artículo antiguo sin ficha 3	1
Guarnizo	OBS-0004	Artículo antiguo sin ficha 4	5
Guarnizo	OBS-0005	Artículo antiguo sin ficha 5	21
Guarnizo	OBS-0006	Artículo antiguo sin ficha 6	24
Guarnizo	OBS-0007	Artículo antiguo sin ficha 7	20
Guarnizo	OBS-0008	Artículo antiguo sin ficha 8	14
Guarnizo	OBS-0009	Artículo antiguo sin ficha 9	24

La respuesta a Marcos: hay 22 artículos con stock y sin ventas en 90 días, todos descatalogados (`activo = 'N'`), por un valor a coste de unos 10.600 €. Y 9 referencias antiguas sin ficha en el ERP que nadie puede vender porque el sistema no las conoce.

REVISIÓN DE ELENA

Muy bien, y con la normalización del SKU aplicada en los dos lados del `JOIN`: si solo la hubieras aplicado en el inventario, los productos del maestro que algún día lleguen con espacios fallarían igual.

Para Marcos, añade algo que no te ha pedido: los descatalogados con stock son una decisión comercial (liquidarlos, devolverlos al proveedor), no de almacén. Copia a Íñigo. Y las 9 referencias sin ficha son un problema de datos: o se dan de alta en el ERP o se dan de baja en el almacén.

PARTE 4

La capa silver

Basta de limpiar en cada consulta: las reglas de limpieza, escritas una vez, en un script que construye tablas limpias para todo el máster.

10. Construir la capa silver

11. Lo mismo, con silver

12. El mismo SQL en otros sitios

10. Construir la capa silver

10.1 El problema que resuelve

Repasa los nueve tickets. En casi todos has repetido lo mismo: el `CASE` de las provincias, el `DISTINCT` de las líneas, el filtro de las fechas imposibles, la normalización del SKU. Eso tiene tres problemas:

1. **Es lento de escribir**, y cada consulta se hace más larga de lo que debería.
2. **Es fácil olvidarse de algo.** Un día haces una consulta deprisa sin el `DISTINCT`, y las ventas salen infladas.
3. **Dos personas pueden limpiar distinto.** Si tú tratas "Santander" como Cantabria y otra persona no, vuestras cifras no coincidirán nunca.

La solución es la capa silver de la arquitectura medallón (Volumen 01, 9.1): escribir las reglas de limpieza **una sola vez**, en un script, y que todas las consultas partan de tablas ya limpias.

> **EL TICKET MJD-210**
>
> Esta parte es el último encargo del volumen. Elena te ha pedido que construyas la capa silver con todo lo aprendido en los tickets, en un script reproducible: `sql/01_construir_silver.sql`. Lo construimos juntos, tabla a tabla.

10.2 Las reglas, primero en castellano

Antes de escribir SQL, las decisiones. Esta tabla irá también al README del repositorio, porque es lo primero que alguien necesita saber para usar silver:

PROBLEMA	REGLA EN SILVER
Provincias escritas de muchas formas	Normalizadas con una tabla de correspondencias. Lo que no encaje: `REVISAR`.
Fichas de cliente duplicadas	Se conservan todas, con una columna `cliente_maestro_id`: la ficha más antigua del mismo NIF.
NIF, CP, teléfono, email	NIF sin guiones ni espacios y en mayúsculas; CP de 5 cifras como texto; teléfono de 9 cifras; email en minúsculas y vacío si no es válido.
Categorías de producto mal escritas	Normalizadas con una tabla de correspondencias.
Líneas duplicadas	Se quita la repetición.
Líneas de pedidos que no existen	Excluidas.
Pedidos de clientes borrados del maestro	Se conservan. El cliente se trata como desconocido en los informes.
Fechas de entrega imposibles	Vacías (`NULL`) y el pedido marcado con `entrega_sospechosa = true`.
SKU del inventario	Al formato del maestro: `CER-0001`.
Importes	Precios en `DECIMAL`. Importe neto de cada línea calculado una vez.

10.3 Las tablas de correspondencias

En lugar de escribir el `CASE` de las provincias en cada sitio, se guarda como una tabla de dos columnas: cómo viene escrito y cómo debe quedar. `VALUES` crea una tabla al vuelo a partir de una lista de filas:

SQL · EN DUCKDB

```
CREATE SCHEMA IF NOT EXISTS silver;

CREATE OR REPLACE TABLE silver.map_provincias AS
SELECT * FROM (VALUES
    ('cantabria',              'Cantabria'),
    ('santander',              'Cantabria'),
    ('asturias',               'Asturias'),
    ('principado de asturias', 'Asturias'),
    ('bizkaia',                'Bizkaia'),
    ('vizcaya',                'Bizkaia'),
    ('burgos',                 'Burgos'),
    ('palencia',               'Palencia')
) AS t(texto_normalizado, provincia);

CREATE OR REPLACE TABLE silver.map_categorias AS
SELECT * FROM (VALUES
    ('cervezas',             'Cervezas'),
    ('cerveza',              'Cervezas'),
    ('refrescos',            'Refrescos'),
    ('aguas',                'Aguas'),
    ('agua',                 'Aguas'),
    ('vinos',                'Vinos'),
    ('licores y destilados', 'Licores y destilados'),
    ('cafe e infusiones',    'Café e infusiones'),
    ('alimentacion',         'Alimentación'),
    ('limpieza e higiene',   'Limpieza e higiene'),
    ('desechables',          'Desechables')
) AS t(texto_normalizado, categoria);
```

La ventaja de tenerlo como tabla: si mañana aparece `'Asturies'`, se añade una fila y no se toca ninguna consulta. Y cualquiera puede ver todas las correspondencias de un vistazo.

10.4 Clientes

SQL · EN DUCKDB

```
CREATE OR REPLACE TABLE silver.clientes AS
WITH fichas AS (
    SELECT *, upper(replace(trim(nif), '-', '')) AS nif_norm
    FROM bronze.clientes
),
maestras AS (
    SELECT nif_norm, min(cliente_id) AS cliente_maestro_id
    FROM fichas
    GROUP BY nif_norm
),
municipios AS (
    -- de las formas en que aparece cada municipio, la que no está toda en mayúsculas
    SELECT lower(municipio) AS clave,
           max(municipio) FILTER (WHERE municipio <> upper(municipio)) AS municipio
    FROM bronze.clientes
    GROUP BY clave
)
SELECT f.cliente_id,
       m.cliente_maestro_id,
       f.cliente_id <> m.cliente_maestro_id                    AS es_ficha_duplicada,
       f.nif_norm                                               AS nif,
       trim(f.nombre_comercial)                                 AS nombre_comercial,
       trim(f.razon_social)                                     AS razon_social,
       f.tipo_cliente,
       coalesce(mu.municipio, f.municipio)                      AS municipio,
       coalesce(mp.provincia, 'REVISAR')                        AS provincia,
       lpad(f.codigo_postal::VARCHAR, 5, '0')                   AS codigo_postal,
       right(regexp_replace(f.telefono, '[^0-9]', '', 'g'), 9)  AS telefono,
       CASE WHEN lower(trim(f.email)) LIKE '%_@_%._%'
            THEN lower(trim(f.email)) END                       AS email,
       f.comercial_id,
       f.fecha_alta,
       f.activo = 'S'                                           AS activo_en_erp,
       f.limite_credito,
       f.descuento_habitual
FROM fichas AS f
JOIN maestras AS m ON m.nif_norm = f.nif_norm
LEFT JOIN municipios AS mu ON mu.clave = lower(f.municipio)
LEFT JOIN silver.map_provincias AS mp ON mp.texto_normalizado = lower(trim(f.provincia));
```

Desmontamos lo nuevo:

DESMONTANDO LA CONSULTA

`SELECT ... FROM fichas f JOIN maestras m ... LEFT JOIN silver.map_provincias mp ...`

`maestras`	Para cada NIF, el `cliente_id` más bajo. Cada ficha se une con la maestra de su NIF, y si no tiene duplicados, es su propia maestra.
`f.cliente_id <> m.cliente_maestro_id`	Una comparación dentro del `SELECT` da `true` o `false`: una columna de tipo `BOOLEAN`.
`municipios`	Para cada municipio (en minúsculas), la forma de escribirlo que no está toda en mayúsculas. Así `SANTOÑA` se convierte en `Santoña` sin escribir una lista.
`coalesce(mp.provincia, 'REVISAR')`	Si la provincia no está en la tabla de correspondencias, el `LEFT JOIN` da `NULL`, y `coalesce` lo convierte en `REVISAR`. Es el `ELSE 'REVISAR'` del 9.4, en versión tabla.
`CASE WHEN ... LIKE '%_@_%._%' THEN ... END`	Un email solo es válido si tiene algo, una arroba, algo, un punto y algo. Si no, el `CASE` sin `ELSE` da `NULL`.
`f.activo = 'S'`	La bandera de texto `'S'` / `'N'` convertida en un `BOOLEAN` de verdad.

10.5 Productos, comerciales e inventario

SQL · EN DUCKDB

```
CREATE OR REPLACE TABLE silver.productos AS
SELECT p.producto_id,
       p.sku,
       p.nombre,
       coalesce(mc.categoria, 'REVISAR')        AS categoria,
       p.subcategoria,
       p.marca,
       p.formato,
       CAST(p.precio_lista   AS DECIMAL(10, 2)) AS precio_lista,
       CAST(p.coste_unitario AS DECIMAL(10, 2)) AS coste_unitario,
       p.iva_pct,
       p.activo = 'S'                           AS activo,
       p.fecha_alta,
       p.fecha_baja
FROM bronze.productos AS p
LEFT JOIN silver.map_categorias AS mc
       ON mc.texto_normalizado = strip_accents(lower(trim(p.categoria)));

CREATE OR REPLACE TABLE silver.comerciales AS
SELECT comercial_id,
       nombre || ' ' || apellidos AS nombre_completo,
       nombre, apellidos, email, zona, puesto, jefe_id, fecha_alta, fecha_baja
FROM bronze.comerciales;

CREATE OR REPLACE TABLE silver.inventario AS
WITH normalizado AS (
    SELECT *, upper(replace(trim(sku), '-', '')) AS sku_norm
    FROM bronze.inventario
)
SELECT n.almacen,
       left(n.sku_norm, 3) || '-' || substr(n.sku_norm, 4) AS sku,
       p.producto_id,
       n.descripcion,
       n.unidades,
       n.ubicacion,
       n.fecha_inventario
FROM normalizado AS n
LEFT JOIN bronze.productos AS p ON p.sku = left(n.sku_norm, 3) || '-' || substr(n.sku_norm, 4);
```

En las categorías, `strip_accents(lower(trim(...)))` hace que `'Alimentacion'`, `'Alimentación '` y `'ALIMENTACIÓN'` acaben siendo la misma clave, `'alimentacion'`. En el inventario, el SKU se reconstruye en el formato del maestro (`CER0007` → `CER-0007`), y se añade el `producto_id` del ERP para que el inventario se pueda unir con todo lo demás por la clave de verdad.

10.6 Pedidos y líneas

SQL · EN DUCKDB

```
CREATE OR REPLACE TABLE silver.pedidos AS
SELECT p.pedido_id,
       p.fecha_pedido,
       CAST(p.fecha_pedido AS DATE)                          AS fecha,
       p.cliente_id,
       c.cliente_maestro_id,
       p.comercial_id,
       p.canal,
       p.estado,
       CASE WHEN p.fecha_entrega >= CAST(p.fecha_pedido AS DATE)
            THEN p.fecha_entrega END                         AS fecha_entrega,
       coalesce(p.fecha_entrega < CAST(p.fecha_pedido AS DATE), false) AS entrega_sospechosa,
       p.almacen,
       CAST(p.gastos_envio AS DECIMAL(10, 2))                AS gastos_envio,
       p.observaciones
FROM bronze.pedidos AS p
LEFT JOIN silver.clientes AS c ON c.cliente_id = p.cliente_id;

CREATE OR REPLACE TABLE silver.lineas_pedido AS
WITH sin_duplicar AS (
    SELECT DISTINCT pedido_id, num_linea, producto_id, cantidad, precio_unitario, descuento_pct
    FROM bronze.lineas_pedido
)
SELECT l.pedido_id,
       l.num_linea,
       l.producto_id,
       l.cantidad,
       CAST(l.precio_unitario AS DECIMAL(10, 2))           AS precio_unitario,
       l.descuento_pct,
       l.cantidad * CAST(l.precio_unitario AS DECIMAL(10, 2)) * (100 - l.descuento_pct) / 100 AS
importe_neto
FROM sin_duplicar AS l
WHERE l.pedido_id IN (SELECT pedido_id FROM bronze.pedidos);
```

Tres decisiones visibles en el código:

- **`fecha_entrega` vacía si es imposible**, y la marca `entrega_sospechosa` para poder encontrar esos pedidos. El `coalesce(..., false)` es necesario porque, para los pedidos sin entrega, la comparación da `NULL` (recuerda 4.6), y queremos `false`.
- **`cliente_maestro_id` ya viene en el pedido.** Para analizar clientes, se agrupa por esa columna, y las fichas duplicadas se suman solas en su maestra. Los pedidos de clientes borrados lo tienen vacío.
- **El importe neto se calcula una vez**, aquí, con los precios en `DECIMAL`. Nadie volverá a escribir la fórmula del descuento (y nadie volverá a equivocarse con ella).

10.7 Todo en un script

Como en el Volumen 01, todo esto va a un fichero, `sql/01_construir_silver.sql`, en el orden de este capítulo: esquema, correspondencias, clientes, productos, comerciales, inventario, pedidos y líneas. El orden importa: `silver.pedidos` usa `silver.clientes`, que tiene que existir antes. Se ejecuta después de cargar bronze:

TÚ ESCRIBES

```
duckdb db/marejada.duckdb < sql/00_cargar_bronze.sql
duckdb db/marejada.duckdb < sql/01_construir_silver.sql
```

Y con eso, cualquiera reconstruye las dos capas desde los ficheros originales en unos segundos.

10.8 Comprobar que silver está bien

Una capa limpia que no se comprueba es una capa en la que se confía a ciegas. Las comprobaciones mínimas, que van en un tercer script, `sql/02_comprobar_silver.sql`:

SQL · EN DUCKDB

```
SELECT 'Clientes: misma cantidad de fichas que en bronze' AS comprobacion,
       (SELECT count(*) FROM silver.clientes) = (SELECT count(*) FROM bronze.clientes) AS ok
UNION ALL
SELECT 'Clientes: ninguna provincia por revisar',
       (SELECT count(*) FROM silver.clientes WHERE provincia = 'REVISAR') = 0
UNION ALL
SELECT 'Productos: ninguna categoría por revisar',
       (SELECT count(*) FROM silver.productos WHERE categoria = 'REVISAR') = 0
UNION ALL
SELECT 'Líneas: ningún (pedido, línea) repetido',
       (SELECT count(*) FROM (SELECT pedido_id, num_linea FROM silver.lineas_pedido
                              GROUP BY ALL HAVING count(*) > 1)) = 0
UNION ALL
SELECT 'Pedidos: ninguna entrega anterior al pedido',
       (SELECT count(*) FROM silver.pedidos WHERE fecha_entrega < fecha) = 0
UNION ALL
SELECT 'Pedidos: 23 entregas marcadas como sospechosas',
       (SELECT count(*) FROM silver.pedidos WHERE entrega_sospechosa) = 23;
```

RESULTADO

```
┌──────────────────────────────────────────────────┬─────────┐
│                    comprobacion                  │   ok    │
│                      varchar                     │ boolean │
├──────────────────────────────────────────────────┼─────────┤
│ Clientes: misma cantidad de fichas que en bronze │ true    │
│ Clientes: ninguna provincia por revisar          │ true    │
│ Productos: ninguna categoría por revisar         │ true    │
│ Líneas: ningún (pedido, línea) repetido          │ true    │
│ Pedidos: ninguna entrega anterior al pedido      │ true    │
│ Pedidos: 23 entregas marcadas como sospechosas   │ true    │
└──────────────────────────────────────────────────┴─────────┘
```

Seis `true`. Si algún día una comprobación sale `false`, es que algo ha cambiado en los datos de origen, y te enteras tú antes que el director comercial. Esto que acabas de escribir a mano es lo que en Fabric y Databricks harás con herramientas de calidad de datos, y es una de las cosas que más se valoran en un ingeniero de datos.

REVISIÓN DE ELENA

Esto es la base de todo lo que vas a hacer en el máster: el modelo en estrella del Volumen 04, Power BI, Fabric y Databricks partirán de aquí. Tres cosas que me gustan: las reglas están escritas antes que el código, bronze sigue intacto, y hay comprobaciones automáticas. Y una que le añadiría: cada regla de limpieza tiene a alguien que la decidió. Pon en el README quién (aunque sea "Elena, 22/10/2026"), porque dentro de un año alguien preguntará por qué Santander cuenta como Cantabria.

11. Lo mismo, con silver

La prueba de que silver merece la pena: los tickets de antes, desde las tablas limpias.

MJD-201, clientes que compran por provincia, contando clientes reales (maestros), no fichas:

SQL · EN DUCKDB

```
SELECT c.provincia,
       count(DISTINCT p.cliente_maestro_id) AS clientes_con_compras_90d
FROM silver.pedidos AS p
JOIN silver.clientes AS c ON c.cliente_id = p.cliente_maestro_id
WHERE p.estado <> 'Cancelado'
  AND p.fecha >= DATE '2026-09-30' - INTERVAL 90 DAY
GROUP BY c.provincia
ORDER BY clientes_con_compras_90d DESC;
```

RESULTADO

```
┌────────────┬──────────────────────────┐
│ provincia  │ clientes_con_compras_90d │
│  varchar   │          int64           │
├────────────┼──────────────────────────┤
│ Cantabria  │                      602 │
│ Asturias   │                      162 │
│ Bizkaia    │                      135 │
│ Burgos     │                       76 │
│ Palencia   │                       52 │
└────────────┴──────────────────────────┘
```

Fíjate en que salen algunos menos que en el MJD-201: son las fichas duplicadas, que ahora cuentan una sola vez. Era la nota al pie de la revisión.

MJD-204, la facturación de contabilidad, para cualquier mes:

SQL · EN DUCKDB

```
WITH portes AS (
    SELECT sum(gastos_envio) AS gastos_envio
    FROM silver.pedidos
    WHERE fecha_entrega >= '2025-03-01' AND fecha_entrega < '2025-04-01'
      AND estado IN ('Entregado', 'Devuelto parcial')
)
SELECT round(sum(l.importe_neto) + any_value(portes.gastos_envio), 2) AS facturacion_contable
FROM silver.pedidos AS p
JOIN silver.lineas_pedido AS l ON l.pedido_id = p.pedido_id
CROSS JOIN portes
WHERE p.fecha_entrega >= '2025-03-01' AND p.fecha_entrega < '2025-04-01'
  AND p.estado IN ('Entregado', 'Devuelto parcial');
```

RESULTADO

```
┌──────────────────────┐
│ facturacion_contable │
│        double        │
├──────────────────────┤
│            1082243.5 │
└──────────────────────┘
```

La misma cifra que contabilidad, sin `DISTINCT`, sin fórmulas de descuento y sin `CASE` de provincias. Los portes se suman aparte, en su propio grano, para no caer en el abanico; el `CROSS JOIN` con esa tabla de una sola fila pone el total de portes al lado de cada línea, y `any_value` coge ese valor, que es el mismo en todas. Cuando en el Volumen 04 construyas el modelo en estrella, ni siquiera eso hará falta.

12. El mismo SQL en otros sitios

Todo lo que has aprendido sirve en Fabric y en Databricks, pero cada motor tiene su **dialecto**: pequeñas diferencias de sintaxis. Las que más te vas a encontrar:

QUÉ	DUCKDB	SQL SERVER / FABRIC WAREHOUSE	DATABRICKS (SPARK SQL)
Las primeras n filas	`LIMIT 10`	`SELECT TOP 10 ...`	`LIMIT 10`
`7 / 2` entre enteros	`3.5`	`3` (división entera)	`3.5`
Concatenar textos	`a || b`	`a + b` o `CONCAT(a, b)`	`a || b` o `concat(a, b)`
Buscar sin mayúsculas	`ILIKE`	No existe: depende de la configuración de la base	`ILIKE`
Convertir tipos	`CAST(x AS INT)` o `x::INT`	`CAST(x AS INT)`	`CAST(x AS INT)` o `x::INT`
Agrupar por todo	`GROUP BY ALL`	No existe	`GROUP BY ALL`
Contar con condición	`count(*) FILTER (WHERE ...)`	`sum(CASE WHEN ... THEN 1 ELSE 0 END)`	`count_if(...)` o `FILTER`
Tipo verdadero/falso	`BOOLEAN`	`BIT` (0 o 1)	`BOOLEAN`
Principio de mes	`date_trunc('month', f)`	`DATETRUNC(month, f)`	`date_trunc('month', f)`

No hace falta memorizarlo. Lo que conviene es saber que existe: cuando en Fabric una consulta que te funcionaba en DuckDB dé error, mira primero en esta tabla. Y la regla de oro para escribir SQL que funcione en todas partes: `CAST` en vez de `::`, `CASE` en vez de `FILTER`, columnas escritas en vez de `GROUP BY ALL`, y divisiones con `100.0`.

APÉNDICES

Para tener a mano

La chuleta de SQL del volumen y lo que viene en el siguiente.

A. Chuleta de SQL

B. Qué viene en el Volumen 03

A. Chuleta de SQL

Consultar

NECESITAS	ESCRIBE
Columnas y alias	`SELECT a, b AS nombre FROM esquema.tabla`
Casi todas las columnas	`SELECT * EXCLUDE (c, d)`
Filtrar	`WHERE ... AND (... OR ...)`
Lista de valores	`WHERE canal IN ('Web B2B', 'Teléfono')`
Un periodo	`WHERE f >= '2025-03-01' AND f < '2025-04-01'`
Patrón de texto	`LIKE '%texto%'` · `ILIKE '%texto%'`
Nulos	`IS NULL` · `IS NOT NULL` · `coalesce(a, b)` · `nullif(a, b)`
Ordenar	`ORDER BY a DESC NULLS LAST`
Valores distintos	`SELECT DISTINCT a`
Decisión	`CASE WHEN ... THEN ... ELSE ... END`

Funciones

TIPO	FUNCIONES
Texto	`trim` `upper` `lower` `replace` `length` `left` `right` `split_part` `strip_accents` `lpad` `regexp_replace` `||`
Núme-ros	`round` `/` `//` `%` `CAST` `TRY_CAST`
Fechas	`CAST(ts AS DATE)` `year` `month` `quarter` `isodow` `date_trunc` `strftime` `f + INTERVAL 30 DAY` `f2 - f1`

Agregar y unir

NECESITAS	ESCRIBE
Contar filas / no nulos / distintos	`count(*)` · `count(a)` · `count(DISTINCT a)`
Agrupar	`GROUP BY a, b` (o `GROUP BY ALL` en DuckDB)
Filtrar grupos	`HAVING count(*) > 1`
Contar con condición	`count(*) FILTER (WHERE ...)`
Unir textos de un grupo	`string_agg(a, ', ')`
Unir conservando la izquierda	`LEFT JOIN t2 ON t2.id = t1.id`
Lo que no tiene pareja	`LEFT JOIN ... WHERE t2.id IS NULL`
Consulta con nombre	`WITH x AS (SELECT ...) SELECT ... FROM x`

NECESITAS	ESCRIBE
Apilar resultados	`UNION ALL`

Crear y exportar

NECESITAS	ESCRIBE
Esquema	`CREATE SCHEMA IF NOT EXISTS silver;`
Tabla desde consulta	`CREATE OR REPLACE TABLE silver.t AS SELECT ...;`
Tabla al vuelo	`SELECT * FROM (VALUES (1, 'a'), (2, 'b')) AS t(id, texto)`
Leer Excel	`read_xlsx('f.xlsx', range = 'A4:E1000')`
Exportar a Excel	`INSTALL excel; LOAD excel; COPY (...) TO 'f.xlsx' WITH (FORMAT xlsx, HEADER true);`
Exportar a CSV	`COPY (...) TO 'f.csv' WITH (HEADER true, DELIMITER ';');`

Las cinco trampas

TRAMPA	CÓMO SE EVITA
`BETWEEN` con fechas que tienen hora	`>=` primer día y `<` primer día del periodo siguiente
`= NULL`	`IS NULL`
`AND` y `OR` mezclados	Paréntesis siempre
Sumar la cabecera después de unir con las líneas	Agregar cada cosa en su grano y unir después
`INNER JOIN` que pierde filas sin avisar	`LEFT JOIN` cuando la izquierda son ventas; comparar recuentos antes y después

B. Qué viene en el Volumen 03

El Volumen 03 (SQL II) parte de la capa silver y añade las herramientas de SQL que distinguen a un analista con experiencia:

- **Subconsultas y `WITH` a fondo**: consultas que se apoyan en otras, paso a paso.
- **Funciones de ventana**: rankings, acumulados, comparar cada mes con el anterior, el último pedido de cada cliente sin agrupar. Es lo que más se pregunta en las entrevistas técnicas de SQL.
- **Fechas avanzadas**: calendarios completos, meses sin ventas, comparaciones año contra año.
- **Rendimiento**: por qué una consulta tarda y cómo leer su plan de ejecución.
- **Tickets nuevos**, entre ellos el que propuso Elena en el MJD-208: ¿los clientes de la web son nuevos o vienen del teléfono?

ANTES DE PASAR AL VOLUMEN 03

Tienes que poder hacer esto sin mirar: filtrar un periodo sin la trampa del `BETWEEN`, contar con las tres formas de `count`, unir dos tablas sabiendo cuántas filas deberían salir, y explicar con un ejemplo por qué los gastos de envío salían multiplicados en el 8.6. Y la capa silver construida y comprobada con tus scripts, porque el Volumen 03 empieza ahí.